MSc Artificial Intelligence
Master Thesis

Plug-and-Play Motion Priors: Enhancing Trajectory Accuracy in Localisation Pipelines

by
Lisann Becker
12558141

October 31, 2026

36 EC
16 January 2025 - 18 July 2025

Supervisor:
Martin R. Oswald
Examiner:
Dimitrios Tzionas
Second reader:
Martin R. Oswald



Universiteit van Amsterdam

Contents

1 Introduction **1**

2 Related Work **4**
2.1 SLAM . . . 4
2.1.1 SplaTAM pipeline . . . 4
2.2 Stochastic Trajectory Prediction with Generative Models . . . 5
2.3 Motion Priors . . . 6

3 Methods **7**
3.1 Backbone Model Architecture and Training . . . 7
3.1.1 Trajectory Representation and Preprocessing . . . 8
3.1.2 Initialiser and Diffusion Model . . . 10
3.1.3 Training . . . 11
3.2 Integration into Localisation Pipelines . . . 12
3.2.1 Motion Prior Construction: Summarising K Predictions via KDE . . . 12
3.2.2 Pose Correction with Motion Prior . . . 12

4 Experiments **14**
4.1 Datasets . . . 14
4.2 Metrics . . . 18
4.2.1 Absolute Trajectory Error (ATE) with Alignment . . . 18
4.2.2 Average Displacement Error (ADE) . . . 19
4.2.3 Final Displacement Error (FDE) . . . 19
4.3 Backbone Experiments . . . 20
4.3.1 Multimodality Verification . . . 20
4.3.2 Noise and Bias Removal . . . 21
4.3.3 Performance in $SE(3)$ . . . 23
4.4 Corrections with Motion Prior . . . 25
4.4.1 Simulated Algorithm Correction . . . 25
4.4.2 SplaTAM Localisation Correction . . . 30
4.5 Ablations . . . 33
4.5.1 Performance of Different Rotation Representations . . . 33

5 Conclusion **35**
5.1 Future Directions . . . 36
5.1.1 Datasets with Stronger Motion Constraints . . . 36
5.1.2 In-Pipeline Motion Prior Regularisation . . . 37

A Appendix **38**

Abstract

Accurate trajectory estimation is essential for robotics, autonomous navigation, and 3D scene reconstruction. However, systems that include localisation branches, such as Simultaneous Localisation and Mapping (SLAM), frequently suffer from drift and noise, especially under challenging conditions that cause sparse or noisy sensory input. To address this, we introduce a diffusion-based *motion prior*, a learned plug-and-play module that refines pose estimates by encoding characteristic motion patterns of specific environments or agent dynamics. Our approach can model a multimodal distribution of plausible future trajectories in SE(3), allowing for complex corrections, all without altering the underlying localisation pipeline.

In simulated localisation scenarios, our motion prior successfully corrects trajectory estimates, reducing drift. When integrated into the state-of-the-art SLAM pipeline SplaTAM, it shows modest performance improvements. Our analysis suggests that the success of the integration depends strongly on how well the learned motion prior aligns with the motion characteristics of the target application. In contexts with relatively regular or constrained motion dynamics, such as autonomous driving, the motion prior may be more effective at reliably modelling expected movement patterns and improving the accuracy of localisation pipelines.

Code is publicly available at: https://github.com/lisannbecker/MotionPrior

Chapter 1

Introduction

Trajectory estimation plays a vital role in robotics, autonomous navigation, and 3D scene reconstruction. Real-world applications enabled by accurate localisation include autonomous driving [19], indoor navigation for service robots [40], augmented reality [37], and search-and-rescue operations after natural catastrophes [21]. For example, in autonomous driving, accurate localisation ensures collision avoidance while in augmented reality, pose estimation is crucial for aligning digital content with the physical world.

However, localisation estimates, as derived within Simultaneous Localisation and Mapping (SLAM) pipelines, often suffer from noise and drift. While drift naturally has more time to accumulate the longer the trajectory, both drift and noise are exacerbated in challenging real-world environments. Poor lighting can degrade RGB data, environments that necessitate sharp turns may lead to sparse observations, and lack of additional sensory information, like inertial or GPS measurements, can all exacerbate problems with drift and noise. To address these limitations, we propose a learning-based *motion prior*, defined as a model that encodes typical patterns of motion learned from historical data and which can correct trajectory estimates. For instance, a car typically follows forward-facing paths with gradual turns and acceleration - such predictable motion patterns can be learned and used to regularise pose predictions. Specifically, our prior is as a frozen plug-and-play module capable of correcting any pose estimate given an input of past poses. In this research, we apply the motion prior to the localisation branch of a state-of-the-art SLAM pipeline, although it could be leveraged for other applications.

Designing our motion prior, we made sure to allow for multimodality in motion. The prior should not only represent the single most likely future trajectory, but given the learned motion constraints should allow for multiple alternative future trajectories. Multi-modality is crucial in real-world scenarios where future motion is influenced by complex factors such as spatial constraints, the kinematics of the agent, or intent. For example, due to the spatial barriers at an intersection, a pedestrian might only have the option to continue walking either right or straight. A non-stochastic model might average these possibilities into an incorrect prediction somewhere in between [14]. On the contrary, a multi-modal approach is able to represent both future paths, right and straight, as distinctly plausible options while ruling out other options such as the space in between.

Recent work in stochastic trajectory prediction, especially diffusion-based methods, can successfully capture such multi-modal distributions over future trajectories for pedestrian motion [24]. Models such as the Motion Indeterminacy Diffusion (MID) [13] and Leapfrog Diffusion Model (LED) [24] generate multiple distinct future trajectories robustly, significantly improving accuracy in uncertain pedestrian motion prediction. However, existing diffusion-based methods have primarily been restricted to 2D (Bird's Eye View) pedestrian settings. Additionally, these

models have been deployed for standalone trajectory prediction tasks, and have not yet been used to develop general motion priors that correct localisation frameworks, particularly those dealing with complex SE(3) trajectory estimation like SLAM.

We propose the idea that a motion prior, trained on a specific motion type, can effectively correct localisation predictions on unseen scenarios of the same motion type. For example, a prior trained from indoor pedestrian motion datasets might be able to correct localisation predictions on other indoor pedestrian datasets. This is under the condition that measurement between datasets is consistent - a handheld sensor might produce different motion dynamics than a back-mounted sensor.

Only recently have motion priors started to gain traction in research. Ma et al. [22] introduced a neural motion prior which modelled camera pose as a continuous function of time. Simply providing temporal regularisation, the prior successfully cut the relative pose error in SplaTAM [17] by 17% and significantly improved rendering quality. Similarly, the TLIO model [20] demonstrated that neural networks trained on inertial data could reliably predict short-term displacement, effectively enhancing classical localisation frameworks. More recently, spline-based approaches such as RESPLE [8] employed recursive Bayesian estimation to achieve real-time and smooth trajectory estimation in continuous-time. However, the existing methods do not leverage advanced generative models such as diffusion models, limiting their capacity to represent multimodal futures and movement typical for a certain type of entity. Our work addresses this gap by introducing a stochastic representation of multiple sampled trajectory predictions, and correcting SLAM localisation predictions against its learned, general motion patterns.

Our goal is to develop this diffusion-based motion prior as a robust, plug-and-play denoising module for SE(3) trajectory estimates within localisation pipelines. Initially focusing on the Gaussian Splatting based SLAM pipeline SplaTAM [17], the long-term ambition is to generalise to other motion estimation frameworks and ultimately generalise to a wider range of motion applications.

To structure the development of the motion prior, we define a central research question capturing the core goal of this work, supplemented by sub-questions that address specific contextual, technical or practical aspects.

Main Research Question *Can we design a learned motion prior based on diffusion models that effectively denoises SE(3) trajectory estimates in SLAM pipelines?*

RQ1 *What are the strengths and limitations of existing stochastic trajectory prediction methods (e.g., LED, MID) for modeling SE(3) motion, and how can they be adapted for pose estimation tasks?*

RQ2 *Can we design a pipeline that effectively learns motion estimates for SE(3) trajectories based on input that is noisy or biased?*

RQ3 *How does applying the motion prior influence localisation performance in both simulated pose estimation algorithms and live SLAM pipelines such as Splat-SLAM, particularly regarding drift reduction?*

RQ4 *Can a diffusion-based motion prior outperform baseline approaches such as the continuous pose function by Ma et al. [22]?*

RQ5 *Which representation of rotation, Lie algebra, quaternions, or 6D continuous representations, leads to the most accurate learning of SE(3) trajectories within the diffusion-based motion prior framework?*

Chapter 2

Related Work

2.1 SLAM

Simultaneous Localisation and Mapping (SLAM) is a widely studied problem in robotics. While we do not develop a new SLAM system, our work focuses on enhancing the performance of existing SLAM systems' pose-estimation modules by incorporating a learned motion prior. SLAM performance has improved dramatically in recent years[25, 26, 6], driven by higher-quality sensors (e.g., high-resolution cameras and advanced IMUs), more robust optimisation algorithms, and increasing computational power that enables real-time dense mapping. Classical approaches such as ORB-SLAM3 [7] achieve robust real-time localisation and mapping across diverse environments. But from its inception until today, long-term drift remains a fundamental challenge in SLAM: small pose estimation errors accumulate over time, especially without loop closures or post-processing [9].

To address drift, many existing systems incorporate inertial measurements or velocity priors to regularise pose estimation [20]. IMUs, for example, constrain orientation and motion estimates, improving robustness in visual-inertial SLAM. Traditional motion initialisation typically assumes constant velocity between estimated poses, which offers basic regularisation but cannot adapt to changing dynamics. Many state-of-the-art SLAM systems like SplaTAM [17] still rely on a constant velocity assumption for pose initialisation and their pose estimation is fully driven by photometric loss minimisation. But even under ideal conditions, SLAM systems are inherently constrained by the resolution of their input data. Limited pixel density and field of view impose a technical upper bound on localisation accuracy. This suggests that augmenting SLAM with an external correction, such as our learned motion prior, could help push performance beyond what is achievable with photometric or geometric consistency alone.

2.1.1 SplaTAM pipeline

SplaTAM [17] is a recent dense RGB-D SLAM framework that leverages Gaussian Splatting to achieve simultaneous camera tracking (localisation) and 3D scene reconstruction (mapping). The following steps are performed iteratively: initially, a dense 3D Gaussian map of the environment is constructed from the first RGB-D frame. Subsequently, incoming frames are processed through camera tracking by minimising photometric (RGB) and geometric (depth) errors between views of the current 3D map and observed frames. These optimisation steps rely on silhouette-masking, through which the loss computations focus on regions that are confidently within the map, making SplaTAM robust towards incomplete data. In the mapping step, SplaTAM continuously updates and expands its Gaussian representation by adding new

Gaussians in underrepresented or newly observed regions, while optimising over a selection of keyframes for computational efficiency.

Despite its advanced components, SplaTAM is still suffering from drift, particularly during sharp turns where observations may be sparse. This challenge is reinforced when SplaTAM is applied to datasets like TUM RGB-D, which contain motion blur due to handheld camera movement. Such blur degrades photometric consistency and makes pose estimation from RGB-D input difficult. In contrast, our motion prior is not affected by visual degradation and can help correct pose estimates under these conditions by comparing the current trajectory to a learned distribution of typical motions.

2.2 Stochastic Trajectory Prediction with Generative Models

Trajectory prediction of agents whose motion is not determined by deterministic algorithms is inherently multimodal. Pedestrians are among the least constrained in terms of motion: they can change direction or stop abruptly and are not restricted by mechanical limits like vehicles. Agents like drones can also more relatively flexibly, however, pedestrian motion is additionally shaped by social context, and environmental cues that are not direct constraints (for example, street signs), resulting in highly variable trajectories. Although pedestrian motion is particularly versatile, architectures of previous approaches have evolved to predict it effectively. Early approaches relied on recurrent models and deterministic outputs [1], but have evolved toward deep generative models that capture uncertainty. Variational Autoencoders (VAEs) [30] and Generative Adversarial Networks (GANs) [12] have been widely adopted for this task [14] as well, though they suffer from issues such as mode collapse and unstable training.

Recent research has demonstrated that diffusion models address these limitations by training more stably and representing complex trajectory distributions more effectively. Unlike VAEs or GANs, denoising diffusion probabilistic models (DDPMs) iteratively refine noisy samples via a sequence of learned denoising steps. In the context of pedestrian motion prediction, MID (Motion Indeterminacy Diffusion) [13] pioneered the idea of treating trajectory uncertainty as a forward diffusion process. Predicting future paths by learning its reverse and gradually resolving uncertainty over time, the model produces coherent trajectory predictions multiple steps into the future. DDPMs' probabilistic nature is suitable for real-world scenarios where multiple distinct future paths are equally plausible.

However, a major bottleneck of diffusion-based prediction is inference latency, which renders them less practical for real-time applications such as SLAM. The Leapfrog Diffusion Model (LED) [24] addresses this by replacing a large portion of the denoising steps with a learned initialiser that outputs diverse trajectory samples directly at an intermediate denoising stage. This technique, which is the architectural inspiration of our motion prior backbone, reduces inference time by up to 30x without sacrificing expressiveness, making diffusion models more viable for near real-time robotic applications.

While not at the center of this research, alternative possible generative approaches for probabilistic trajectory modelling, such as normalising flows [28] and flow-matching methods [18], offer advantages. Normalising flows use invertible transformations to model data distributions with exact likelihoods, while flow-matching learns time-continuous probability flows via ODE solvers. Compared to diffusion models, flow-based methods allow for exact likelihood evalua-

tion and faster inference. However, they can be harder to scale to high-dimensional trajectory data like SE(3) due to invertibility constraints. Therefore, in this work, we adopt a diffusion model for its flexibility and robustness in predicting possible trajectories in SE(3).

2.3 Motion Priors

Recent research that may be interpreted as a motion prior, models motion as a continuous function of time [22]. As classic SLAM pipelines typically estimate discrete poses at each frame, they fail to capture the inherent smoothness of real-world motion. Early solutions used either splines or Gaussian Process regression to interpolate trajectories [2], while a recent approach by Ma et al. [22] simply learns implicit representations of pose over time. The new approach models the camera pose as a continuous SE(3) function using a lightweight MLP, optimised with mapping objectives. This results in temporally smoothed trajectories that enables estimating poses at arbitrary timestamps, and supports low-DOF motion representations that can be interpreted as motion prior. Integrated into SLAM systems like SplaTAM [17], this type of temporal regularisation significantly reduces relative pose error.

Another approach, RESPLE [8], embeds 6-DoF motion in cubic B-splines estimated via a recursive Extended Kalman Filter (EKF), efficiently fusing LiDAR and IMU data into smooth trajectories. Building on this idea of motion regularisation, several recent efforts have explored incorporating learned priors into EKF-based visual-inertial SLAM. Traditional filtering methods such as EKF and factor-graph optimisation [7] fuse inertial and visual data but often struggle to capture environment-dependent or history-aware motion patterns. In contrast, learned motion models like TLIO [20] show that neural networks can predict short-term IMU-based displacements, which can then be integrated into classical estimation pipelines to improve robustness and accuracy.

While such approaches have shown to mitigate drift, they cannot represent the full range of plausible futures and consider long-term motion dependencies. Our stochastic, diffusion-based motion prior addresses these limitations in two ways: it learns a multimodal SE(3) distribution which allows pose estimates to be corrected toward several plausible futures conditioned on learned motion dynamics and it leverages sequences of past poses to encode long-term temporal dependencies for more informed corrections. Building on pedestrian-focused diffusion methods in 2D, we extend these ideas to full SE(3) trajectories and new agent classes (e.g. cars) by modifying the Leapfrog architecture [24] and training with a regularisation objective inspired by Ma et al. [22]. We aim to integrate this learned prior model as a plug-and-play pose correction module within localisation pipelines such as SplaTAM's [17], improving performance without changing the existing architecture.

Chapter 3

Methods

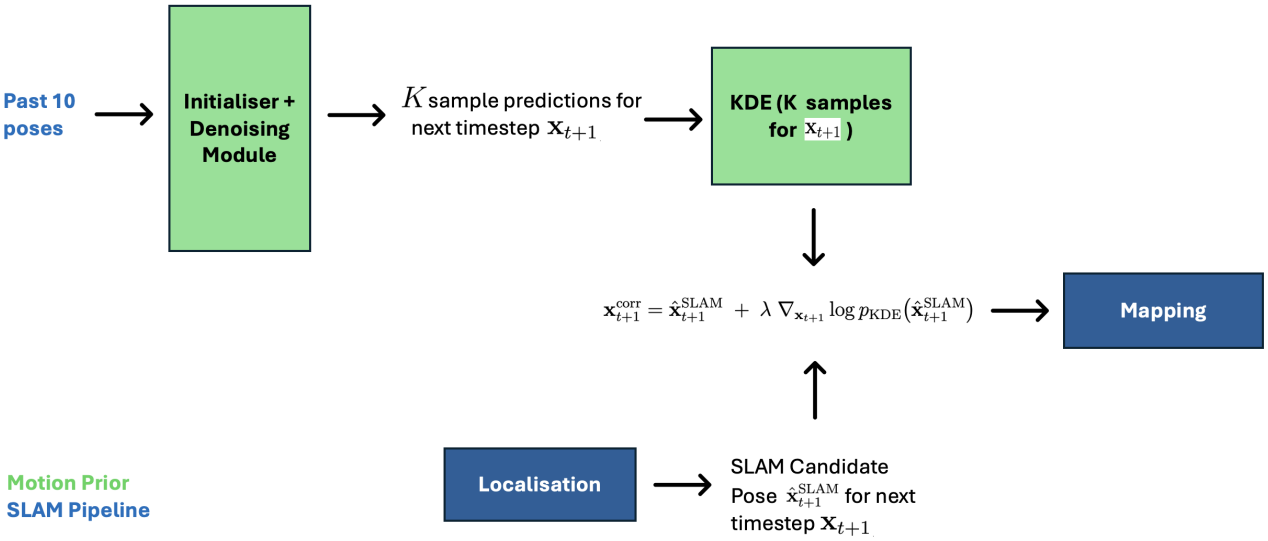


Figure 3.1: SLAM pipeline (Section 2.1.1) predicts the next pose $\hat{\mathbf{x}}_t$. Independently, our model (Section 3.1.2) generates K candidate trajectories for $\hat{\mathbf{x}}_t$ from the past 10 relative SE(3) poses (Section 3.1.1). These samples are summarised into a (potentially multimodal) KDE (Section 3.2.1), whose gradient is used to update the SLAM estimate and produce the corrected pose (Section 3.2.2).

Our learned motion prior captures typical motion dynamics by building a probabilistic density from multiple future trajectory samples that we produce from our frozen diffusion-based backbone model. We first train this backbone diffusion model to generate diverse, plausible future poses conditioned on past observations. Once trained, we freeze the backbone and integrate it into the live localisation pipeline: using only the past 10 poses as input, at each time step it generates K candidate future trajectories, which we summarise via a Kernel Density Estimate (KDE). The resulting KDE defines our motion prior and produces a corrected SLAM pose estimate by nudging the raw pose toward regions of higher probability. Figure 3.1 shows this architecture and the sections describing its components.

3.1 Backbone Model Architecture and Training

This section outlines how our backbone prior architecture is trained and used to generate pose samples for the KDE-based motion prior. First, we describe trajectory representation and preprocessing, specifically explaining how we embed trajectories as relative poses with

velocities. Next, we explain the various rotation representations explored in our experiments and why they might influence trajectory representation. Then we introduce the backbone model architecture, including an initialiser module that accelerates inference and a diffusion denoising network. Lastly, we describe the training procedure, including loss definition and training hyperparameters.

3.1.1 Trajectory Representation and Preprocessing

We support trajectory data representations that include 2D translation only, 3D translation only, and 3D translation combined with various rotation representations: Lie algebra (3 parameters), quaternions (4 parameters), and continuous 6D rotation representations.

Each trajectory is represented only via *relative* poses and their velocities (first-order differences). We omit absolute poses as global orientation (e.g. north vs. south) or the total distance travelled before our sequence begins should be irrelevant to our local trajectory prediction. Therefore, global motion data makes the learning task unnecessarily complex and potentially undermines performance by diverting attention from relevant local motion patterns.

$$\left\{\mathbf{x}_t^{\text{raw}}\right\}_{t=1}^{T_{\text{obs}}} \subset \mathbb{R}^D \tag{3.1}$$

are the raw, absolute poses over T_{obs} observed time steps, where $\mathbf{x}_t^{\text{raw}} = (\mathbf{p}_t,\, \rho_t)$ splits into translation $\mathbf{p}_t \in \mathbb{R}^d$ and rotation ρ_t.

Relative translation and velocity To compute relative translations, we subtract the last observed pose $\mathbf{p}_{T_{\text{obs}}}$ from every other pose in the trajectory, past and future. Hence, the trajectory is recentred such that the final observed pose is always at the origin $(0x,\, 0y,\, 0z)$. This recentring is visible in the trajectory visualisations of this paper.

Formally, relative translations are computed as:

$$\Delta\mathbf{p}_t = \mathbf{p}_t - \mathbf{p}_{T_{\text{obs}}}, \quad t = 1, \ldots, T_{\text{obs}} + T_{\text{pred}}. \tag{3.2}$$

where T_{pred} denotes the number of future prediction steps.

Consequently, future trajectories effectively "start from zero". For example, if the agent moves into a positive direction, past positions would take on negative values, the last observed position is exactly at $0, 0, 0$, and future predictions would take on positive values.

Next, we calculate velocities $\mathbf{v}_t$, defined as first-order differences of these relative translations:

$$\mathbf{v}_t = \Delta\mathbf{p}_t - \Delta\mathbf{p}_{t-1}, \quad \mathbf{v}_1 = \mathbf{0}. \tag{3.3}$$

We choose to provide velocities to the model explicitly to aid effective learning of patterns conditional on velocity. For example, we observed in the KITTI dataset that slower velocities often correlate with turns, whereas higher velocities are more frequent on straight trajectories. Explicitly providing velocity information enables the model to efficiently exploit such relationships, especially using smaller datasets 4.2.

Relative rotation We support multiple rotation representations as each address different challenges with representing 3D orientations. Our goal is to explore how representation choice impacts performance in our specific trajectory prediction scenarios and for different datasets. For example, no single parameterisation is without caveats. Lie-algebra representations can suffer discontinuities (e.g. a jump at 360°), quaternions avoid those singularities but are over-parameterised by one dimension so their 4D outputs must be normalised to unit norm, and continuous 6D representations embed rotation matrices into $\mathbb{R}^6$, ensuring smoothness but requiring an orthogonality constraint (e.g. via Gram–Schmidt) during training [4, 11]. The comparative performance of these rotation representations will be analysed in detail in section 4.5.1.

Similarly to translations, we align rotations to the last observed frame by computing relative orientations with respect to $\rho_{T_{\text{obs}}}$, shifting the final observed orientation to the identity. This factors out global rotation. Just as with translation, recentring orientation removes information and hence, bias on global pose, eliminating the need for the network to factor out this arbitrary global orientations, and improving generalisation.

To capture relative orientation ρ_t^{rel}, we compute

$$\rho_t^{\text{rel}} = \mathsf{RelRot}\big(\rho_t,\, \rho_{T_{\text{obs}}}\big), \tag{3.4}$$

where RelRot is implemented as:

- **Lie–algebra (3D):** $\log\big(\exp(\phi_t)\, \exp(-\phi_{T_{\text{obs}}})\big) \in \mathbb{R}^3$.
- **Quaternions (4D):** $\mathbf{q}_{T_{\text{obs}}}^{-1} \otimes \mathbf{q}_t \in \mathbb{R}^4$.
- **Continuous 6D:** convert the 6D rep. to $R \in SO(3)$, form $R_{T_{\text{obs}}}^{\top} R_t$, then back to 6D.

$\phi_t, \phi_{T_{\text{obs}}} \in \mathbb{R}^3$ represent the axis–angle vectors of the rotations at times t and T_{obs}; $\mathbf{q}_t, \mathbf{q}_{T_{\text{obs}}} \in \mathbb{R}^4$ represent unit quaternions encoding those same rotations, and $\otimes$ denotes quaternion multiplication; and $R_t, R_{T_{\text{obs}}} \in SO(3)$ represent the corresponding 3×3 rotation matrices.

Per-timestep Pose Embedding We stack relative translation $\Delta\mathbf{p}_t \in \mathbb{R}^d$, relative rotation $\rho_t^{\text{rel}} \in \mathbb{R}^{\dim(\rho)}$, and velocity $\mathbf{v}_t \in \mathbb{R}^d$ into a single feature vector per time step:

$$\mathbf{x}_t \;=\; \big[\, \Delta\mathbf{p}_t \,\big\|\, \rho_t^{\text{rel}} \,\big\|\, \mathbf{v}_t \big] \;\in\; \mathbb{R}^D, \quad D = d + \dim(\rho) + d. \tag{3.5}$$

For example, when $d = 3$:

$$D = \begin{cases} 3+3+3 = 9, & \text{Lie algebra (3D);} \\ 3+4+3 = 10, & \text{quaternions (4D);} \\ 3+6+3 = 12, & \text{continuous 6D rotation.} \end{cases}$$

3.1.2 Initialiser and Diffusion Model

The backbone model follows the Leapfrog Diffusion Model (LED) architecture [24], extended to handle SE(3) poses instead of 2D translation, variable-length past and future trajectories, dynamic trajectory sample counts K and omitting absolute pose values from the pose embeddings 3.1.1. Varying the number of generated candidate trajectory samples K allows us to trade capturing high degrees of multi-modality with computational efficiency - which one we prioritise depends on the dataset. For example, a dataset with relatively simple and unimodal trajectories may require fewer samples to cover possible future poses than a more complex, highly multimodal dataset. Therefore, in a simpler dataset, we can use fewer trajectory samples K to sufficiently cover future trajectory options and prioritise inference speed.

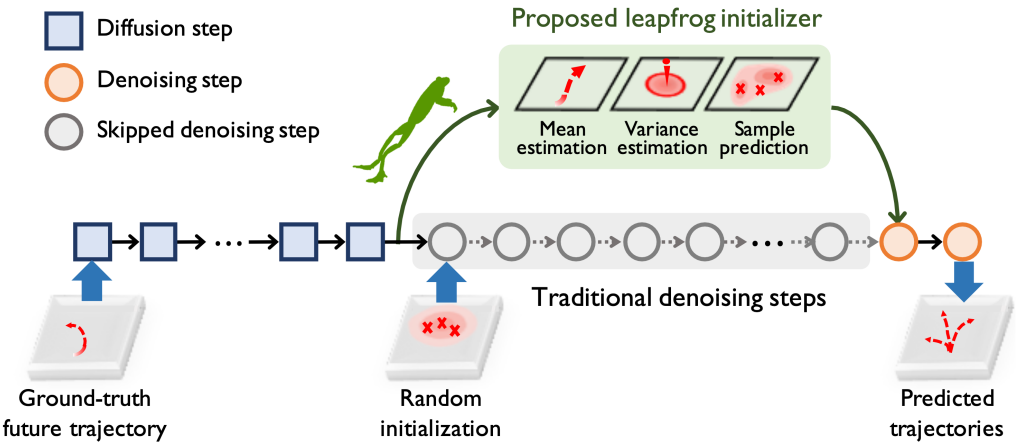


Figure 3.2: Overview of training the Leapfrog Diffusion (LED) backbone: after the forward diffusion steps (blue squares), the trainable initialiser (green box; see Section 3.1.2) estimates the mean trajectory, variance, and sample offsets to produce K intermediate trajectory candidates at step τ. These candidates are then refined by a small number of transformer-based denoising steps (orange circles; see Section 3.1.2) to yield the final multimodal trajectory predictions. The visualisation is copied from Mao et al. [24].

Figure 3.2 summarises the architecture of the backbone model. At inference, given a set of observed past poses $\{\mathbf{x}_t\}_{t=1}^{T_{\text{obs}}}$, the initialiser produces K intermediate trajectory candidates at diffusion step τ, which the denoiser then refines into the final multimodal predictions.

Initialiser Instead of initialising from pure Gaussian noise, the pipeline employs a trainable Initialiser model, directly generating intermediate distributions of future trajectories at time step τ. This distribution is modelled explicitly by predicting the following components:

- **Mean Trajectory** μ_θ: The central, most probable future path.
- **Variance** σ_θ: Overall uncertainty or spread around the mean.
- **Sample Positions** $S_{\theta,k}$: Normalised offsets around the mean trajectory, which introduce multi-modality into the predictions.

The Initialiser generates candidate trajectories $\hat{\mathbf{x}}_t^{(k),\tau}$ as:

$$\hat{\mathbf{x}}_t^{(k),\tau} = \mu_\theta + \sigma_\theta \, S_{\theta,k}, \quad k = 1, \ldots, K, \tag{3.6}$$

The mean μ_θ, variance σ_θ, and normalised sample positions $S_{\theta,k}$ are outputs of dedicated modules conditioned on past trajectories.

Denoising Module The denoising module refines each intermediate candidate $\{\hat{\mathbf{x}}_t^{(k),\gamma+1}\}$ toward the final prediction via a small number of learned diffusion steps. Conditioned on the past context, it iteratively removes noise as follows:

$$C = f_{\text{context}}(\{\mathbf{x}_t\}_{t=1}^{T_{\text{obs}}}), \tag{3.7}$$

$$\epsilon_\theta^\gamma = f_\epsilon\big(\hat{\mathbf{x}}_t^{(k),\gamma+1},\, C,\, \gamma+1\big), \tag{3.8}$$

$$\hat{\mathbf{x}}_t^{(k),\gamma} = \frac{1}{\sqrt{\alpha_\gamma}}\Big(\hat{\mathbf{x}}_t^{(k),\gamma+1} - \frac{1-\alpha_\gamma}{\sqrt{1-\bar{\alpha}_\gamma}}\,\epsilon_\theta^\gamma\Big) + \sqrt{1-\alpha_\gamma}\,z, \quad z \sim \mathcal{N}(0, I). \tag{3.9}$$

Here, f_{context} encodes temporal and spatial information from the observed trajectory $\{\mathbf{x}_t\}_{t=1}^{T_{\text{obs}}}$, and f_ϵ estimates the residual noise at each diffusion step $\gamma+1$. Repeating (3.9) for $\gamma = \tau-1, \ldots, 0$ produces the final set of refined trajectories $\{\hat{\mathbf{x}}_t^{(k),0}\}_{k=1}^K$.

3.1.3 Training

The backbone model is trained by minimising a combined distance and uncertainty loss, which balances prediction accuracy with diversity. Extending the model to SE(3) poses, we added separate losses to handle translation and rotation. Translation errors were evaluated using the standard Euclidean distance, while rotation errors were measured via geodesic distances on SO(3). These loss components were then normalised and combined into a joint training objective.

We minimise a weighted sum of two terms:

$$\mathcal{L} = w_{\text{dist}}\,\mathcal{L}_{\text{dist}} + \mathcal{L}_{\text{unc}}\,. \tag{3.10}$$

Distance loss $\mathcal{L}_{\text{dist}}$: the best-of-$K$ prediction error in SE(3), combining translation (Euclidean) and rotation (geodesic) distances.

Uncertainty loss $\mathcal{L}_{\text{unc}}$: encourages accurate calibration of modelled uncertainty.

$$\mathcal{L}_{\text{dist}} = \min_k \frac{1}{T}\sum_{t=1}^{T}\Big(w_{\text{trans}}\|\mathbf{p}_t^{\text{gt}} - \hat{\mathbf{p}}_t^{(k)}\|_2^2 \;+\; w_{\text{rot}}\, d_{\text{SO(3)}}\big(\rho_t^{\text{gt}},\, \hat{\rho}_t^{(k)}\big)^2\Big) \tag{3.11}$$

$$\mathcal{L}_{\text{unc}} = \frac{1}{K\,T}\sum_{k=1}^{K}\sum_{t=1}^{T}\Big(e^{-\sigma_{\theta,t}^2}\big\|\mathbf{x}_t^{\text{gt}} - \hat{\mathbf{x}}_t^{(k)}\big\|_2^2 \;+\; \sigma_{\theta,t}^2\Big) \tag{3.12}$$

where $\mathbf{x}_t = (\mathbf{p}_t, \rho_t)$ is the SE(3) pose (translation + rotation), $\{\hat{\mathbf{x}}_t^{(k)}\}_{k=1}^K$ are the K predicted trajectories, $d_{\text{SO(3)}}()$ is the geodesic rotation distance, $\sigma_{\theta,t}^2$ is the predicted variance at time t, and $w_{\text{trans}}, w_{\text{rot}}, w_{\text{dist}}$ balance translation accuracy, rotation accuracy, and the overall distance term, respectively.

Training used an input trajectory size of 10 timesteps, a diffusion process consisting of 150 steps, of which 145 were skipped by the initialiser and the remaining $\tau = 5$ were denoised by the denoising module with a linear beta schedule (β_{start} = 1e-4, β_{end} = 5e-2), a learning rate of 1e-3, batch sizes of 64 for both training and testing, and a step-based learning rate scheduler with decay every 8 epochs (decay factor 0.5). We experimented with varying numbers of prediction samples K and future timesteps T_{pred}, which are reported in our results.

We conducted all experiments on NVIDIA A100 GPU nodes, and consumed a total of 93,562:05 SBUs.

3.2 Integration into Localisation Pipelines

With the backbone model trained and frozen, we plug it into a localisation pipeline as a post-processing plug-and-play correction module to refine the pipeline's estimated trajectories towards typical movement patterns.

With the backbone model trained and frozen, we integrate it into a localisation system like SLAM as a lightweight, post-processing module. At each timestep, the model generates K future pose predictions based on recent past motion. These are summarised into a KDE, which defines a motion prior. The independent SLAM pose estimate is then refined by taking a gradient step in the direction of increasing KDE log-likelihood.

3.2.1 Motion Prior Construction: Summarising K Predictions via KDE

To build our motion prior, we fit a kernel density estimate (KDE) to the K predicted samples for a given timestep. This KDE is implemented as a multivariate Gaussian mixture with diagonal covariance, using our own `CustomKDE` class. This allows us to explicitly control the bandwidth per dimension, in contrast to libraries such as SciPy, which fail to adapt KDE bandwidth to the metric scale of the translation and rotation.

Given the K samples at a particular timestep (e.g. K=30 alternative future point estimate predictions), we first filter out statistical outliers: a point is removed if it has fewer than one neighbour within 1.5× the median pairwise distance, using a local radius. This avoids overly broad KDE tails caused by isolated predictions which the backbone model tends to produce under small training data size.

The KDE is then fitted to the filtered samples. Its bandwidth (per dimension) is tuned manually as described and justified in our results 4.4.

This KDE defines a probability distribution over the K candidate poses; in the next section, we explain how its log-density gradient is used to correct the localisation estimate of the separate localisation pipeline.

3.2.2 Pose Correction with Motion Prior

We leave the localisation pipeline whose predictions we correct unchanged and use our learned motion prior as a plug-and-play post-processing step. At each timestep t, while the localisation pipeline, let us assume a SLAM pipeline, produces its own localisation candidate $\hat{\mathbf{x}}_{t+1}^{\text{SLAM}}$, we predict K future trajectory samples using the frozen diffusion-based backbone model, conditioned on the past T_{obs} poses:

$$\left\{\hat{\mathbf{x}}_{t+1:t+T_{\text{pred}}}^{(k)}\right\}_{k=1}^{K} \sim p_{\text{prior}}\left(\mathbf{x}_{t+1:t+T_{\text{pred}}} \mid \mathbf{x}_{t-T_{\text{obs}}+1:t}\right). \tag{3.13}$$

While we generate a full T_{pred}-step trajectory, in SLAM, we typically only correct the next pose. We nonetheless predict farther ahead as considering the larger time horizon improves the first pose prediction. So after predicting K pose predictions for each timestep t, we then extract the K candidates at $t+1$, to ultimately correct the immediate next pose:

$$\left\{\hat{\mathbf{x}}_{t+1}^{(k)}\right\}_{k=1}^{K}. \tag{3.14}$$

Next, we build the KDE as in 3.2.1, which defines a probability distribution over the K samples for $\mathbf{x}_{t+1}$:

$$p_{\text{KDE}} = \frac{1}{K}\sum_{k=1}^{K}\mathcal{N}\left(\mathbf{x}_{t+1};\, \hat{\mathbf{x}}_{t+1}^{(k)},\, \mathbf{H}\right), \tag{3.15}$$

where $\mathbf{H}$ is our diagonal bandwidth matrix.

Now we compute the gradient of the log-density at SLAM's own next-step point estimate $\hat{\mathbf{x}}_{t+1}^{\text{SLAM}}$:

$$\nabla_{\mathbf{x}_{t+1}} \log p_{\text{KDE}}\big(\hat{\mathbf{x}}_{t+1}^{\text{SLAM}}\big). \tag{3.16}$$

Lastly, we get the final corrected pose $\mathbf{x}_{t+1}^{\text{corr}}$ via a gradient ascent step in the direction of the KDE's log-density gradient, scaled by the stepsize λ.

$$\mathbf{x}_{t+1}^{\text{corr}} = \hat{\mathbf{x}}_{t+1}^{\text{SLAM}} \; + \; \lambda \, \nabla_{\mathbf{x}_{t+1}} \log p_{\text{KDE}}\big(\hat{\mathbf{x}}_{t+1}^{\text{SLAM}}\big), \tag{3.17}$$

If we correct multiple future timesteps, as in our simulated algorithm experiments 4.4.1, we apply the exact same KDE gradient procedure independently at each step $t+2, t+3, \ldots$.

Chapter 4

Experiments

4.1 Datasets

Sequences from each dataset were divided into subtrajectories based on the chosen window size; for example, for 10 past and 20 future poses we split the sequences into non-overlapping subtrajectories of length 30. Additionally, our custom dataloader supports options such as relative vs. absolute poses, optional normalisation, overlapping subtrajectories 4.1, selection of pose dimensionalities 3.1.1, batch assembly, reproducible splits with fixed seeds, and customisable train/validation/test proportions.

Our datasets and their characteristics are shown in table 4.1. Note that sensor placement directly influences the motion priors learned. For example, despite all datasets reflecting pedestrian motion, backpack-mounted pedestrian data in Oxford Spires shows different joint motion dynamics than handheld captures in Newer College or TUM RGB-D. And even two handheld motion patterns can be distinct depending on how the camera is carried - for example, photographs of the Newer College recording session show them using a device held stably at around head height while TUM RGB-D was recorded from hip height.

Dataset	Pose	Sensor	Motion	GT Measurement
Oxford Spires [32]	Quaternion	Backpack	Pedestrian (outdoor)	ICP-to-TLS map
Newer College [27]	Quaternion	Handheld	Pedestrian (outdoor)	ICP-to-LiDAR map
KITTI Odometry [10]	4×4 SE(3)	Car roof	Driving	LiDAR+IMU odometry
TUM RGB-D [31]	Quaternion	Handheld	Pedestrian (indoor)	Motion-capture system
Synthetic	4×4 SE(3)	n/a	Straight & curved	n/a

Table 4.1: For each dataset (1) original pose format, (2) sensor mounting, (3) motion type, and (4) GT measurement.

Dataset	Train	Val/Test
Oxford Spires [32]	605	76
Newer College [27]	768	96
KITTI Odometry [10]	615	77
TUM RGB-D [31]	487	61
Synthetic (All Variants)	8000	1000

Table 4.2: Number of subtrajectories used for training and validation/testing (validation and testing sample sizes) for each dataset.

In table 4.3 we report which experiments we utilised the datasets for, while 4.2 shows that we worked with limited dataset sizes, with the exception of our synthetic datasets. Table 4.4 shows which sequences of the datasets we selected - they were chosen based on comparability. For example, we selected only outdoor sequences from Newer College, streamlining environmental conditions that might affect motion patterns. Secondly, we only selected TUM RGB-D sequences featuring rapid handheld motion, hence with homogenous motion patterns.

Dataset	Motion Prior	Simulated Corr.	SplaTAM Corr.
Oxford Spires	✓	✗	✓
Newer College	✓	✗	✓
KITTI Odometry	✓	✓	✗
TUM RGB-D	✓	✓	✓
Synthetic	✓	✓	✗

Table 4.3: Dataset usage across experiments: columns shows whether (1) a motion prior was trained on the dataset, (2) simulated pose predictions were corrected using the prior, and whether (3) live SLAM predictions (SplaTAM pipeline) were post-processed with the prior.

Dataset	Sequences
KITTI Odometry	00, 01, 02, 03, 04, 05, 06, 07, 08, 09, 10
Newer College	nc-quad-easy, nc-quad-medium, nc-quad-hard, nc-stairs, nc-park, gt_math_easy, gt_math_medium, gt_math_hard
Oxford Spires	observatory-quarter-01, observatory-quarter-02, bodleian-library-02
TUM RGB-D	freiburg1_desk, freiburg1_desk2, freiburg1_room

Table 4.4: Sequences used from each dataset for training and evaluation. For KITTI, all sequences with available ground truth were used (00–10). For all other datasets, we selected sequences with comparable environmental and measurement conditions that affect the motion dynamics our motion prior learns.

Synthetic Datasets

Next to standard datasets, we generate a synthetic dataset to investigate the motion prior's ability to model multi-modality 4.3.1 and ability to correct noisy and biased sequences 4.3.2 under controlled conditions with a simulated algorithm.

Synthetic Fork Dataset. The **Synthetic Fork** dataset is designed to explicitly verify that our backbone model, and therefore the motion prior, can capture a truly multimodal distribution. Each trajectory's past $\mathbf{x}_{t-T_{\text{obs}}+1:t}$ is a straight segment. The future trajectory $\mathbf{x}_{t+1:t+T_{\text{pred}}}$ then either turns right or left with equal probability (50%), but never continues straight. The radius of the curve is sampled from a uniform distribution between 3.5 and 5.0 meters, introducing some variability. The K samples produced by the backbone model should capture this variability by producing samples within this radius range, but not produce samples outside of the two branches. Figure 4.1 visualises ten example trajectories from this dataset.

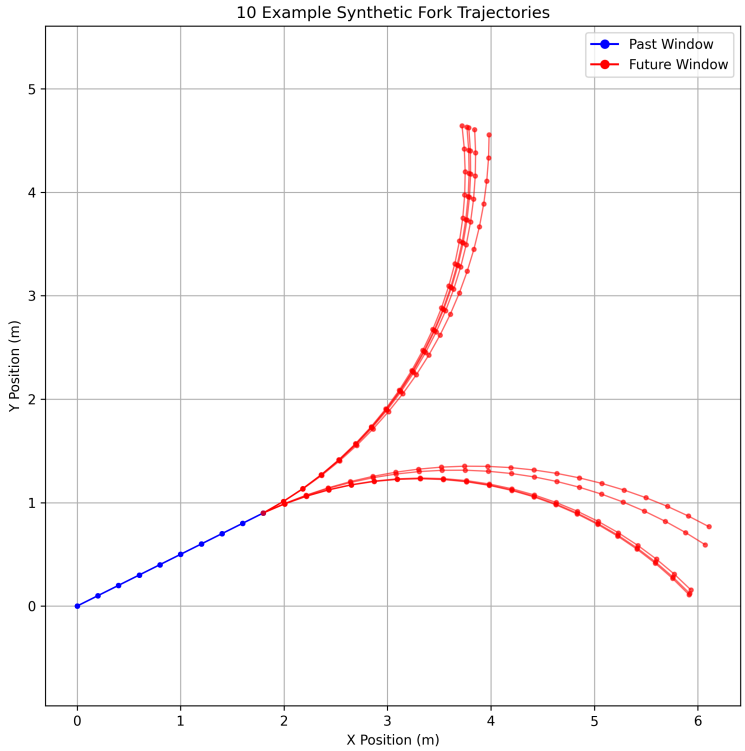


Figure 4.1: Ten example trajectories from the Synthetic Fork dataset, specifically five left-turning and five right-turning samples. All trajectories share the same straight past segment $\mathbf{x}_{t-T_{\text{obs}}+1:t}$, but diverge in the future $\mathbf{x}_{t+1:t+T_{\text{pred}}}$ with slightly different curvature.

Synthetic Noise and Bias Dataset. In our **Synthetic Noise** dataset, each trajectory consists of a noisy past and a clean ground-truth future. We simulate two types of GT motion patterns: (1) **Straight** and (2) **Right Curve** GT trajectories. We add three different types of stochastic or systematic noise (bias) to the past poses to simulate realistic measurement errors and drift:
(1) **Independent Random Noise** - zero-mean noise added independently at each timestep, does not accumulate over time;
(2) **Random Walk Noise** - random noise is added relative to the previous (already noisy) pose, causing it to accumulate over time and simulate drift, which is more realistic since previous errors do not simply disappear;
(3) **Asymmetric Drift** - a consistent rightward bias is added at each step, combined with random noise, simulating systematic error such as sensor misalignment.

These GT motion types and noise conditions combine into the following six distinct datasets:

- Straight GT + Independent Random Noise
- Straight GT + Random Walk Noise
- Straight GT + Asymmetric Drift
- Right Curve GT + Independent Random Noise
- Right Curve GT + Random Walk Noise
- Right Curve GT + Asymmetric Drift

In all conditions, the initial past pose $\mathbf{x}_{t-T_{\text{obs}}+1}$ remains unperturbed and the ground-truth future segment remains noise-free which allows us to test how well the prior can learn the GT despite noisy measurements 4.3.2. The experiments section includes a figure of all six conditions 4.3.

Overfitting Setting

To better understand the upper performance limits of our motion prior models, we include an **overfitting condition** for each dataset. Instead of splitting sequences into non-overlapping subtrajectories, this condition uses a sliding window with an offset of one pose, meaning each window is shifted by only one pose relative to the previous one. As a consequence, training and test subtrajectories can share nearly identical data, and can differ as little as one timestep. Such overlap intentionally introduces overfitting, and lets us test the best possible performance our model can achieve while not having to generalise. Concretely, we leverage this setting for the simulated algorithm 4.4.1 and SplaTAM 4.4.2 corrections, to identify how well the motion prior corrects trajectories under idealised circumstances.

4.2 Metrics

4.2.1 Absolute Trajectory Error (ATE) with Alignment

To evaluate the quality of predicted trajectories, we compute the *Absolute Trajectory Error* (ATE). This metric measures the deviation of a predicted trajectory from the ground-truth trajectory over all time steps. Since predicted trajectories may experience global shifts (e.g., translation offsets or rotation drift) that do not reflect actual errors in the motion pattern, we first apply rigid alignment using Umeyama's method [33]. This ensures that the ATE reflects differences in trajectory *shape* rather than global misalignment.

Let K be the number of samples per input, T the number of future steps, and D the pose dimension. Denote the k-th predicted and ground-truth poses at time t by

$$\hat{\mathbf{x}}_t^{(k)} \in \mathbb{R}^D, \quad \mathbf{x}_t \in \mathbb{R}^D, \quad k = 1, \dots, K,\ t = 1, \dots, T. \tag{4.1}$$

We measure translation error, hence we extract the translation components:

$$\hat{\mathbf{p}}_t^{(k)} = \hat{\mathbf{x}}_t^{(k)}[:3], \quad \mathbf{p}_t = \mathbf{x}_t[:3]. \tag{4.2}$$

Rigid Alignment Each predicted trajectory $\{\hat{\mathbf{p}}_t^{(k)}\}_{t=1}^T$ is rigidly aligned to the ground-truth trajectory $\{\mathbf{p}_t\}_{t=1}^T$ by solving

$$\min_{R,t} \sum_{t=1}^{T} \left\| R\big(\hat{\mathbf{p}}_t^{(k)} - \bar{\hat{\mathbf{p}}}^{(k)}\big) + \bar{\mathbf{p}} - \mathbf{p}_t \right\|_2^2, \tag{4.3}$$

where

$$\bar{\hat{\mathbf{p}}}^{(k)} = \tfrac{1}{T} \sum_{t=1}^{T} \hat{\mathbf{p}}_t^{(k)}, \quad \bar{\mathbf{p}} = \tfrac{1}{T} \sum_{t=1}^{T} \mathbf{p}_t.$$

Denote the aligned predictions by $\tilde{\mathbf{p}}_t^{(k)}$.

Best-of-K Selection Compute the mean translation error e_k for each sample:

$$e_k \;=\; \frac{1}{T} \sum_{t=1}^{T} \left\| \tilde{\mathbf{p}}_t^{(k)} - \mathbf{p}_t \right\|_2, \quad k = 1, \dots, K. \tag{4.4}$$

Select

$$k^* = \arg\min_k e_k, \tag{4.5}$$

and let the best trajectory be $\tilde{\mathbf{p}}_t^* = \tilde{\mathbf{p}}_t^{(k^*)}$.

ATE Computation The final ATE is the root-mean-squared translation error (RMSE) of the best aligned trajectory:

$$\mathrm{ATE} = \sqrt{\frac{1}{T} \sum_{t=1}^{T} \left\| \tilde{\mathbf{p}}_t^* - \mathbf{p}_t \right\|_2^2}. \tag{4.6}$$

4.2.2 Average Displacement Error (ADE)

The Average Displacement Error (ADE) measures the average point-wise Euclidean distance between the predicted and ground-truth trajectories over the entire predicted trajectory. In our experiments, for each trajectory and time window, we compute:

$$\text{ADE} = \frac{1}{T} \sum_{t=1}^{T} \|\hat{\mathbf{p}}_t^{k^*} - \mathbf{p}_t\|_2$$

This metric captures how closely the entire predicted trajectory follows the ground truth on average.

4.2.3 Final Displacement Error (FDE)

The Final Displacement Error (FDE) instead focuses only on the endpoint, computing the Euclidean distance between the final predicted position and the final ground-truth position:

$$\text{FDE} = \|\hat{\mathbf{p}}_T^{k^*} - \mathbf{p}_T\|_2$$

This metric emphasises the accuracy of the final position prediction.

Limitations of ADE and FDE

We report ADE and FDE as they are standard metrics in trajectory prediction. However, they have shortcomings we want to overcome by supplementing the ATE metric. Both only consider the geometric distances between predicted and actual trajectories, without regard for the qualitative shape or feasibility of the trajectories. Because they work with averages over positions, they can assign similar scores to predictions that differ substantially in direction and distribution of error per pose.

For example, FDE measures only the distance at the last time step, so two trajectories that end at the same point but follow completely different paths can yield identical FDE scores. More concretely, for FDE, errors at different time steps can cancel each other out. For example, consider a two-step prediction horizon: if the first predicted pose overshoots the ground truth by 10 cm in the x direction, and the second predicted pose undershoots by 10 cm, then the final displacement error (FDE) would misleadingly indicate zero mean error, even though neither individual prediction is actually close to the ground truth.

Because of these issues, we complement ADE and FDE with the ATE metric described earlier. Unlike ADE and FDE, ATE with alignment removes errors caused solely by overall pose misalignment, and instead focuses on the comparing shape of the predicted and GT trajectory.

4.3 Backbone Experiments

4.3.1 Multimodality Verification

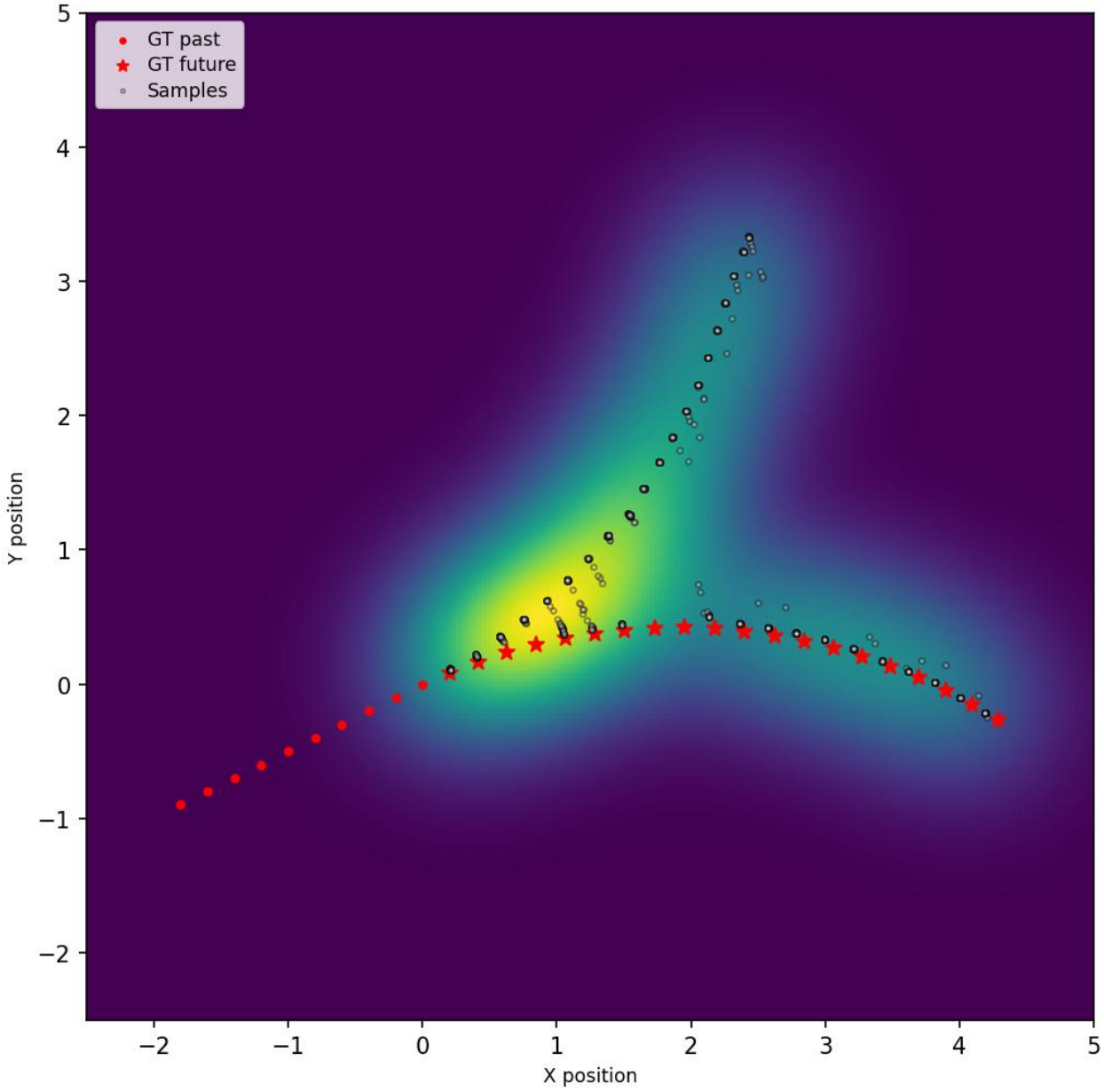


Figure 4.2: Samples from our backbone model on the Synthetic Fork dataset. All filtered K samples at every future timestep are overlaid, and the per-timestep KDE densities have been summed to produce the green-blue heatmap where brighter green shows higher density of samples. Note that although shown in a single image for better oversight, both the samples and the KDEs are actually computed independently at each timestep.

In our truly multimodal Synthetic Fork dataset, we see that our backbone model successfully, although not perfectly, captures multimodality. The backbone model produces approximately equal samples for the two branches, matching the 50/50 data distribution. Moreover, the samples somewhat cover the variability we introduced to the radius of the left and right curves, although most samples by far fall around the mean of the two branches. This concentration around the average radius might result from the Initialiser model's mean and variance param-

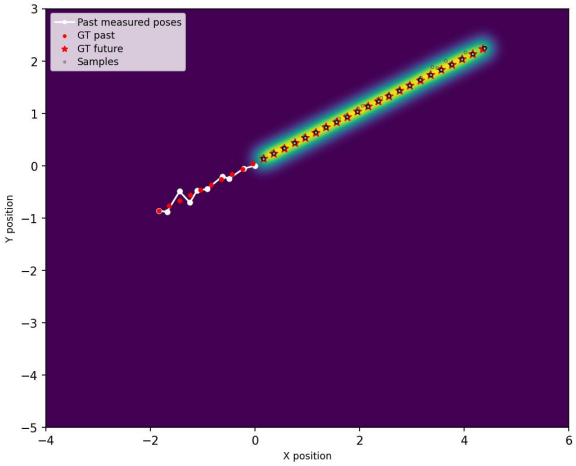
eterisation and its training losses. The Initialiser explicitly predicts a mean trajectory and a single variance per dimension, and the "best-of-K" distance loss together with the uncertainty penalty might discourage samples from straying too far: so most offsets collapse toward the high-probability mean of each branch. Lastly, with only $\tau = 5$ denoising steps that weren't skipped, the Denoiser has limited ability to compensate for the Initialiser and correct samples to the outer radius. In summary, our multimodality experiment demonstrates that the learned motion prior successfully models *coarser* variability in a multimodal dataset without generating invalid trajectories like collapsing to a single mode.

4.3.2 Noise and Bias Removal

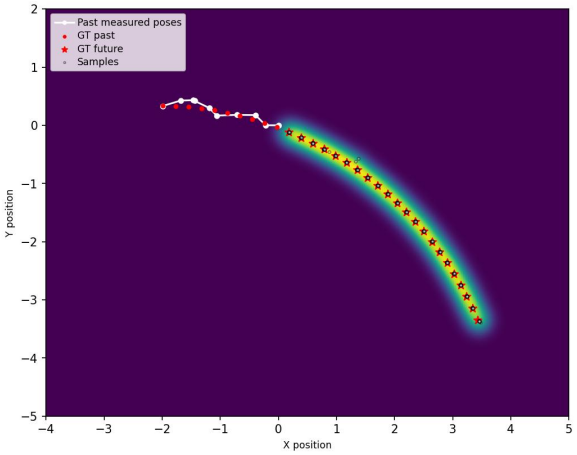
Next, we evaluate the ability of our backbone model to handle noisy or biased measurements using the **Synthetic Noise** dataset 4.1. Each trajectory within this dataset contains past poses that are corrupted by either independent random noise (a and b in Figure 4.3), accumulating random walk noise (c and d in Figure 4.3, or asymmetric right-biased drift combined with random noise (e and f in figure 4.3). The dataset encompasses two types of ground-truth (GT) trajectories: straight (a, c, and e 4.3) and right-curved motions (b, d, and f 4.3), resulting in six distinct scenarios. The model needs to predict a clean, noise-free future trajectory. This means that if the model learns to factor out noise and bias, it should be able to predict the future trajectory (near) perfectly.

In each of these scenarios, we generated 10,000 training sequences and the model predicted 20 future poses based on 10 past poses. Despite only ever observing the noisy or biased past poses, and never the ground-truth past, the model successfully learns to correct the synthetic measurement errors. As shown in Figure 4.3, the sampled predictions very closely match the ground-truth future trajectories. They are spread minimally, reflecting that the future trajectory is noise-free.

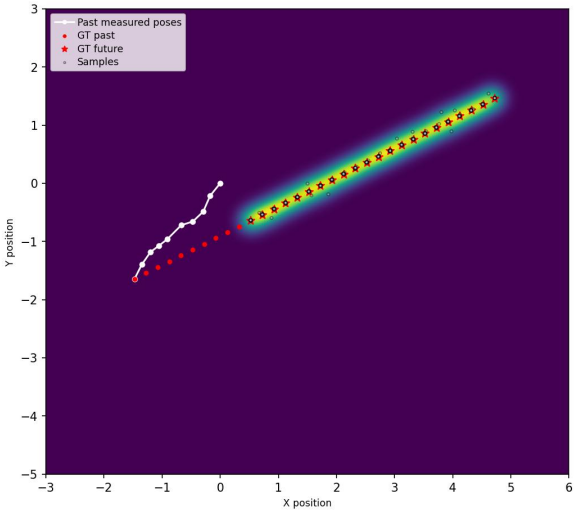
This ability is relevant for real-world robotics pipelines, where sensor measurements or pose estimates from algorithms may contain systematic errors or noise. Our preliminary results suggest that, provided sufficient training data, the motion prior can learn these specific patterns of measurement error or bias. Therefore, for sensors or algorithms with known errors, explicitly incorporating such patterns during training would enable our model to factor them out in its predictions. While these results show potential for bias and noise correction, we have not yet integrated this learned correction into our experiments with the SplaTAM pipeline 4.4.2 as we lack known error characteristics.



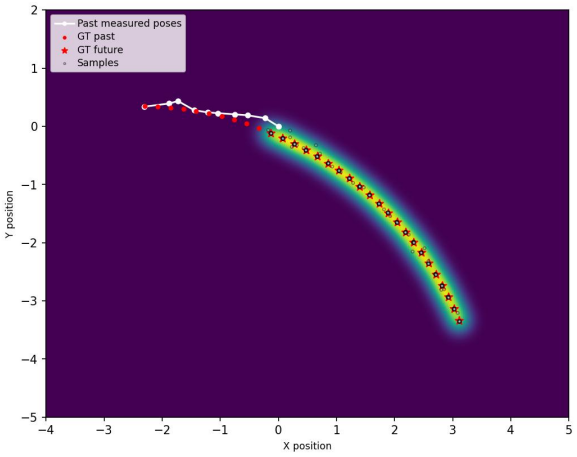
(a) Indep. Random Noise, Straight GT



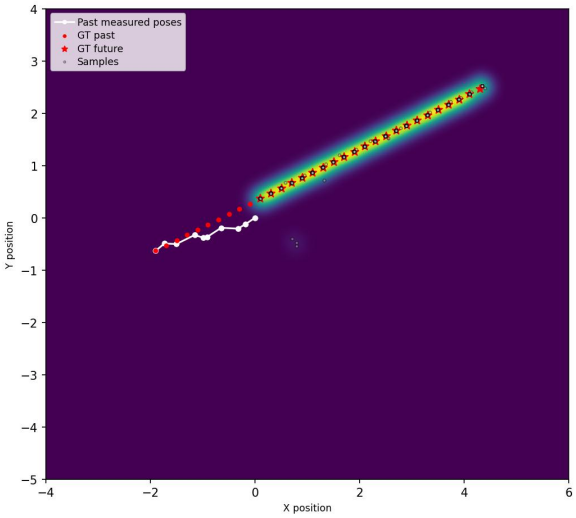
(b) Indep. Random Noise, Right-Curve GT



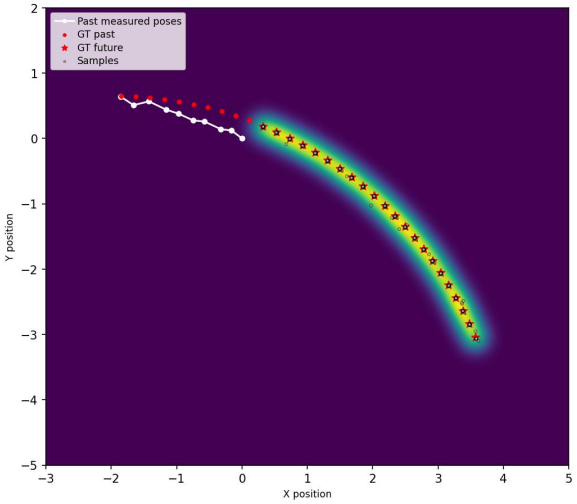
(c) Random Walk Noise, Straight GT



(d) Random Walk Noise, Right-Curve GT



(e) Asymmetric Drift, Straight GT



(f) Asymmetric Drift, Right-Curve GT

Figure 4.3: All pose samples K closely match the the ground truth, showing that the backbone model learned to factor out noise and bias. We added and example KDE on top of these samples (green-blue heatmap). Note that, although shown together here for better overview, both the sampling and KDE estimation are computed independently at each timestep.

4.3.3 Performance in $SE(3)$

Dataset	Translation Error (m) ATE/ADE/FDE
Oxford Spires	0.0913/0.0795/0.1393
Newer College	0.2094/0.1663/0.3170
KITTI Odometry	1.2238/0.8284/2.3818
TUM RGB-D	0.0127/0.0109/0.0232
Synthetic Custom	0.3100/0.1407/0.1224

Table 4.5: Translation errors for 20-step predictions with $K = 30$ on each dataset. We report the best-performing rotation representation found and discussed in our ablation study 4.5.1. Specifically, quaternions were optimal for Oxford Spires, Lie algebra for Newer College, and 6D continuous representation for KITTI, TUM RGB-D, and Synthetic Custom. Hence, each result reflects the lowest achievable trajectory error for our backbone model on that dataset.

Dataset	3D Path (m)	ATE per Step (m)	Ours ATE (m)	ATE/Path (%)
Oxford Spires	1.243 22	0.0046	0.0913	7.3
Newer College	2.679 03	0.0105	0.2094	7.8
KITTI Odometry	18.378 84	0.0612	1.2238	6.7
TUM RGB-D	0.063 56	0.0006	0.0127	20.0
Synthetic Fork	7.886 59	0.0155	0.3100	3.9

Table 4.6: Translational ATE of $K = 30$ in context of the total 3D path length traversed over 20 future steps: per-step ATE, overall ATE, and the corresponding error percentage.

Across our real-world datasets (Oxford Spires, Newer College, KITTI, and TUM RGB-D), ATE ranges from about 0.01m up to 1.22m for 20-step predictions (Table 4.5). When we put these errors in context by dividing by the total 3D path length traversed - between 0.06m and 18.4m depending on the dataset - the resulting error percentages span roughly 3.9% to 20.0% (Table 4.6). Notably, the synthetic Fork dataset, trained on 8,000 sequences 4.2, achieves the lowest normalised error at 3.9%.

Context and Baselines As we predict purely from past poses over a short subwindow (10 input poses to 20-step output poses), and we predict from pose-only, the reported performance is not directly comparable to SLAM or VO systems that use additional sensors and optimise over entire trajectories. To the best of our knowledge, there is little prior work that tackles pure pose-only forecasting on these robotics benchmarks.

For more contextualisation of the performance, we report the performance of pedestrian benchmarks in our appendix A.3. However, while use poses only, with an input of 10 and an output of 20 like our method, they differ in significant ways that make them not comparable with our results.

First, most pedestrian-trajectory models predict only 2D (x, y) displacements, whereas we operate in SE(3), estimating translation in three axes (including vertical motion). So our errors include an additional z-axis component and orientation error. Second, many pedestrian benchmarks are multi-agent: they jointly forecast all nearby people and model social interactions, whereas our motion prior is a single-agent model. Third, pedestrian datasets are substantially larger often including tens of thousands of trajectories (e.g. 32,500 in NBA) while our largest

real dataset contains fewer than 800 sequences, which affects the quality of learned motion patterns. Last, pedestrian papers hardly report the total path length traversed during prediction, so their ADE/FDE numbers cannot easily be contextualised: a 0.3m error on a 2m path is very different from the same error on a 20m path.

4.4 Corrections with Motion Prior

In our correction approach, we want to leverage the motion prior - representing typical motion of the prior train data agent in its environment - as a correction to an independent localisation pipeline, such as the localisation branch of a SLAM pipeline. Instead of only applying our motion prior correction to a real-world pipeline, we first correct a simulated algorithm as proof of concept. The correction lies under the assumption that the ground-truth pose lies somewhere between the estimates provided by an imperfect algorithm and the predictions generated by our multi-modal KDE-based motion prior. We now evaluate whether our method can correct pose estimates into this intermediate region, and bring the pose estimate closer to the ground truth.

4.4.1 Simulated Algorithm Correction

We tested the correction capabilities of the motion prior on a simulated algorithm across several conditions, primarily using subsets of the KITTI dataset.

Condition	Prior Data	SLAM Data	Simulated Algorithm
1	KITTI (excl. seq. 03)	KITTI seq. 03	Linear model
2	KITTI 80% train split	KITTI 10% test split	Linear model
3	KITTI 80% train split overfit	KITTI 10% test split overfit	Linear model

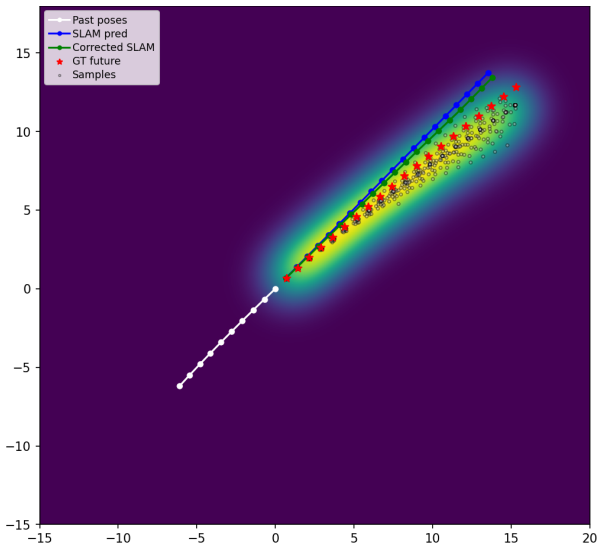
Table 4.7: Overview of our three simulated-algorithm experiments. For each condition, we list (1) the KITTI data used to train the motion prior ("Prior Data"), (2) the KITTI data on which the simulated pose estimates are corrected ("Correction Data"), and (3) the dummy localisation algorithm (a simple linear-trajectory model) whose outputs are post-processed by our KDE-based motion prior. The SplaTAM equivalent of this table is Table 4.9.

Condition	σ	λ	**Backbone** *affects potential correction* ATE/ADE/FDE	**Algorithm** *before motion prior* ATE/ADE/FDE	**Corrected** *after motion prior* ATE/ADE/FDE
	1.4	1.6	0.7834/0.6030/0.8767	1.0138/0.8363/1.9064	**0.9643**/**0.7946**/1.8293
	1.4	2.4	—	—	0.9786/0.8125/**1.8097**
1	1.4	2.0	—	—	0.9684/0.8007/1.8179
	1.0	2.0	—	—	1.0683/0.8954/1.9226
	0.6	2.0	—	—	1.6544/1.4717/2.7353
	1.4	1.6	1.2062/0.8201/2.3849	1.7833/1.4368/3.4586	**1.7598**/**1.4100**/**3.4673**
	1.4	2.4	—	—	1.7832/1.4321/3.4952
2	1.4	2.0	—	—	1.7693/1.4190/3.4797
	1.0	2.0	—	—	1.8670/1.4971/3.9135
	0.6	2.0	—	—	2.1787/1.8268/4.3675
	1.4	1.6	0.4374/0.2801/1.4998	1.7833/1.4368/3.4586	1.1724/0.8649/2.5048
	1.4	2.4	—	—	**1.0725**/**0.7977**/**2.2765**
3	1.4	2.0	—	—	1.1005/0.8061/2.3654
	1.0	2.0	—	—	1.2436/0.9882/2.4408
	0.6	2.0	—	—	1.5503/1.3309/2.5325

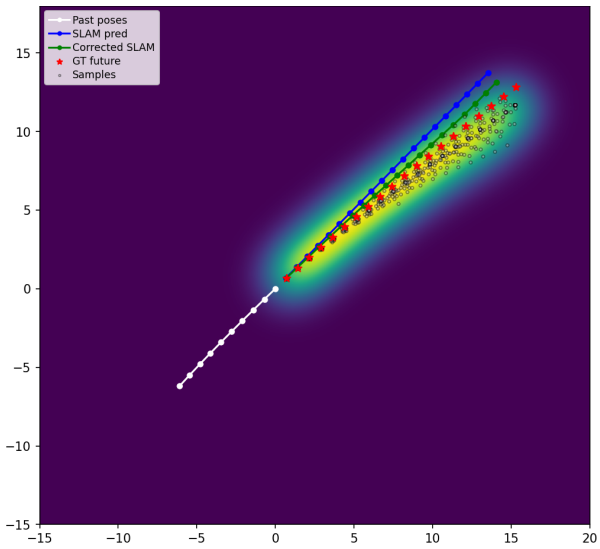
Table 4.8: Performance of correction of the simulated algorithm over 20 future steps using $K = 30$ samples and a quaternion rotation representation. Initial pose errors (ATE/ADE/FDE) before motion-prior correction are shown in the "Algorithm" column; corrected errors are in the "Corrected" column. For each condition, the best corrected ATE, ADE, and FDE values are highlighted in bold.

The KDE-based motion prior correction consistently improves performance across all three test conditions. The largest gain appeared in Condition 3, where overlapping trajectories provided more prior data, yielding a corrected ATE of 1.0725 m down from 1.7833 m. The results demonstrate that selecting appropriate KDE bandwidth (σ) and update stepsize λ is critical: poorly parameter choice worsened the prediction when σ does not match the motion-scale or λ overshoots. Optimal correction requires matching σ to the dataset's trajectory scale; varying λ depending on how accurately the motion prior predicts trajectories and hence how much trust is placed in it versus the original prediction; and choosing λ small enough to avoid overshoot over multiple future steps. When these criteria are met - especially in datasets with consistent prior quality - the same σ and λ settings deliver reliable improvements across conditions. The following sections illustrate these conditions.

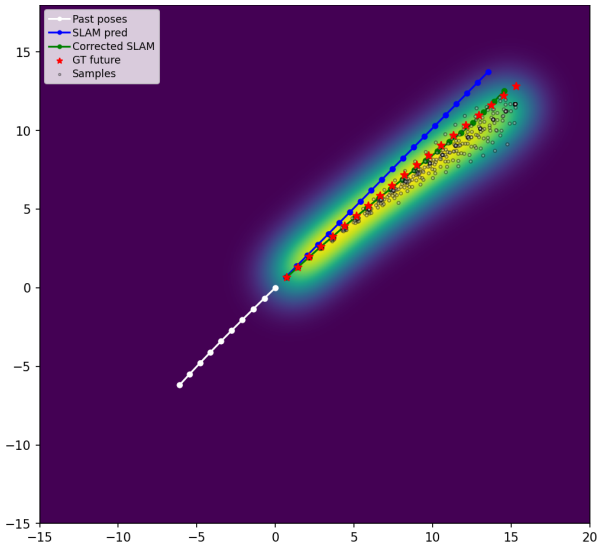
Adequate Correction vs. Overcorrection Figure 4.4 shows that for the same trajectory, bandwidth σ and stepsize λ determine whether we achieve an adequate correction or an overcorrection. While an excessively λ leads to update steps that overshoot, an excessive σ produces log-density gradients so steep that the correction likewise overshoots.



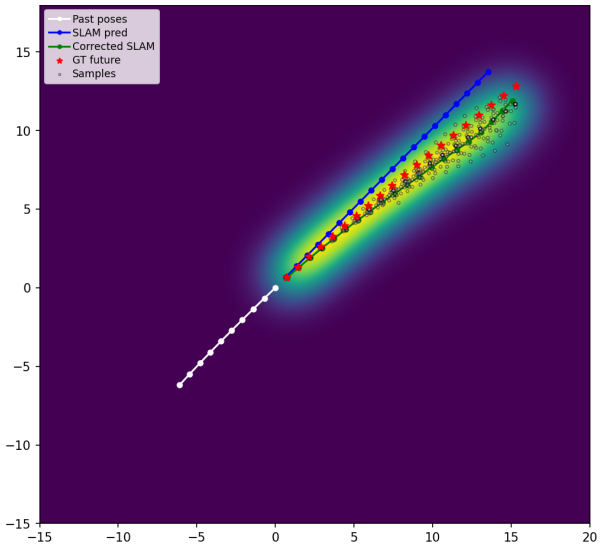
(a) 1.4 σ, 0.4 λ



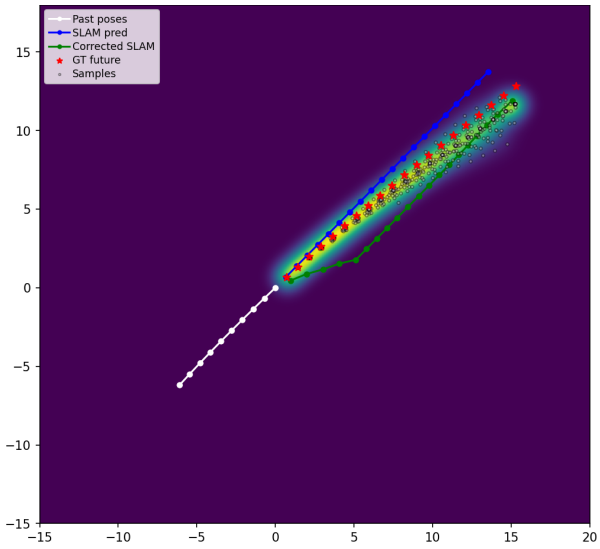
(b) 1.4 σ , 0.8 λ



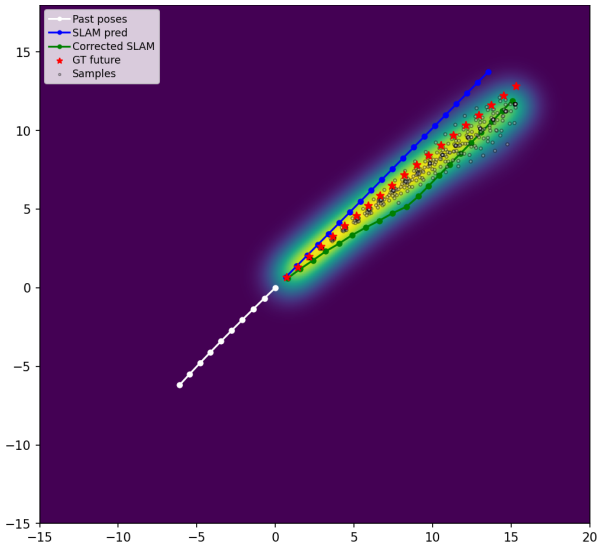
(c) 1.4 σ, 1.8 λ



(d) 1.4 σ, 2.4 λ - overcorrection through λ



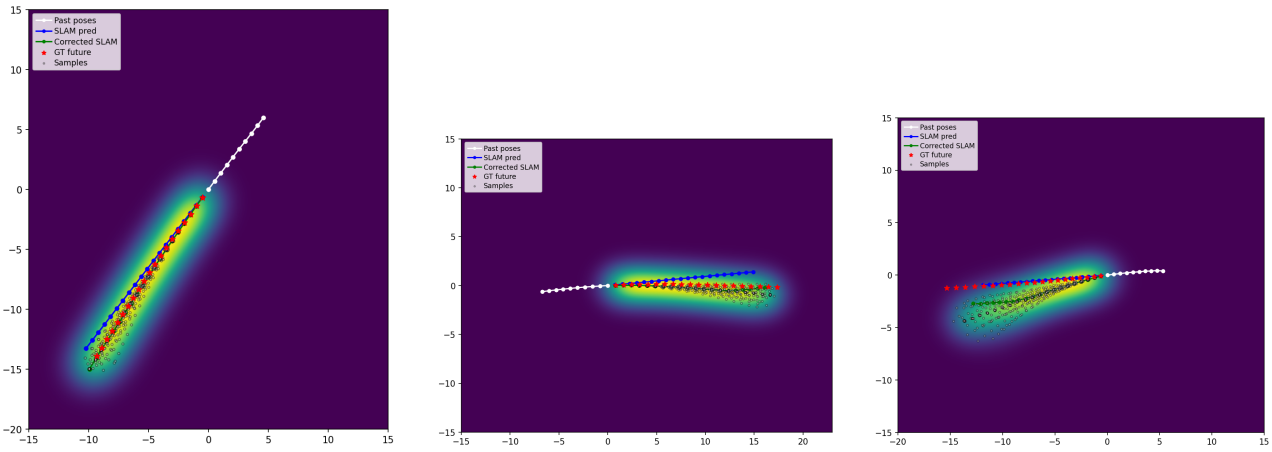
(e) 0.6 σ, 1.8 λ - overcorrection through σ



(f) 1.0 σ, 1.8 λ - overcorrection through σ

Figure 4.4: Correction outcomes on a single KITTI trajectory under varying KDE prior bandwidth (σ) and update stepsize (λ).

Poor vs. High Quality of Prior Prediction The quality of the prior affects optimal correction parameters. When the prior accurately reflects the true underlying motion patterns, larger correction steps (higher λ values) become feasible. Conversely, if pulling the initial predictions too far to the prior predictions means overstepping the ground truth, we need more conservative correction parameters. Essentially, the correction parameters need to reflect our trust in the uncorrected algorithm versus the prior as they define where we want to land in between. Figure 4.6 illustrates that the same parameters might lead to an appropriate correction for a trajectory with a high quality prior (image a) but a mild (image b) to drastic (image c) overcorrection if the prior is far off from the GT.



(a) Adequate Correction (b) Slight Overcorrection (c) Drastic Overcorrection

Figure 4.5: Correction outcomes using ($\sigma = 1.4$, $\lambda = 2.0$) for trajectories with varying prior quality: (a) adequate correction when the motion prior closely matches the true motion; (b) slight overcorrection with a moderate prior quality; (c) drastic overcorrection when the prior far overshoots the ground truth.

Leaving the Bandwidth The KDE bandwidth σ puts an upper limit on how far the correction can move the prediction. This only becomes relevant when the algorithm's predictions fall entirely outside the support of the KDE. In such cases, the correction plateaus regardless of stepsize λ. This limitation capped our correction performance in the simulated algorithm experiment, as the update magnitude no longer increased with distance once the algorithm left the bandwidth. Figure 4.6 shows three examples where the algorithm predictions drifted so far off from the KDE that it hit the correction ceiling.

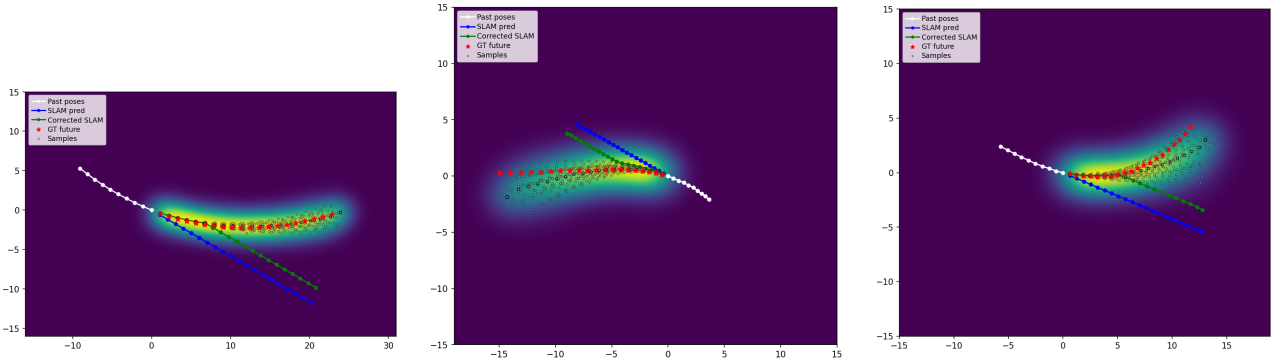


Figure 4.6: Examples of cases where the simulated algorithm's predictions fall outside the KDE. Once outside the bandwidth σ, the correction no longer increases with distance.

This phenomenon is relevant in our simulated experiments, where we predict many steps into the future and the divergence between the algorithm and prior grows over time. In contrast, our SLAM correction focused on the single next timestep. While the SLAM algorithm prediction theoretically could still fall outside of the KDE support, this is far less likely for a next pose prediction, making this limitation less of a concern in practice.

4.4.2 SplaTAM Localisation Correction

Condition	λ	σ	ATE (cm)
Existing SplaTAM Variants			
SplaTAM baseline [17]	—	—	3.58
SplaTAM + PoseNet [22]	—	—	**2.99**
Our Motion-Prior Experiments			
Pedestrian prior on Oxford Spires & Newer College	0.001	0.004	3.74
Trained on TUM RGB-D excl. desk1	0.001	0.004	3.54
Trained on TUM RGB-D excl. desk1	0.005	0.004	3.52
Trained on TUM RGB-D excl. desk1	0.005	0.015	4.03
Trained on TUM RGB-D desk1 (overfit)	0.005	0.015	3.58
Trained on TUM RGB-D desk1 (overfit)	0.005	0.004	3.49

Table 4.9: Final ATE RMSE (cm) for baseline SplaTAM variants versus our motion-prior corrections on the Freiburg desk1 sequence of TUM RGB-D. Existing methods are listed first, followed by our experiments; the best overall ATE is shown in bold.

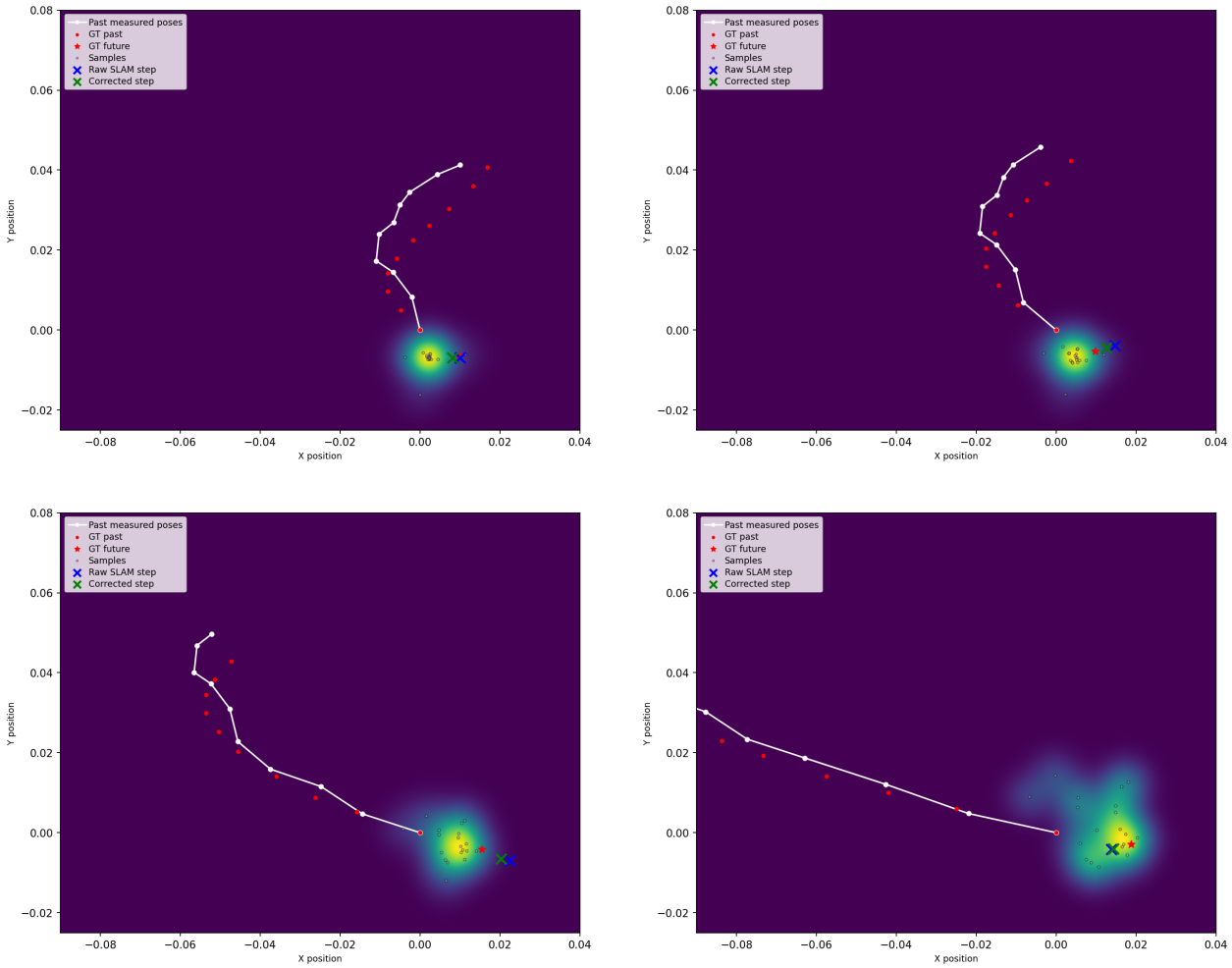


Figure 4.7: Motion-prior correction examples on TUM RGB-D desk1 with $\lambda = 0.001$ and $\sigma = 0.004$. Illustrate how the KDE-based prior trained on TUM RGB-D (all sequences except desk1) pulls the original prediction into a higher-density region.

We apply our motion prior correction to localisation estimates by SplaTAM on the TUM RGB-D desk1 sequence. Figure 4.7 shows examples of these corrections. Our goal is to evaluate whether our motion prior can effectively correct these pose estimates by leveraging general knowledge about likely motion patterns it learned from its training data. We trained priors under different conditions. First, we used a pedestrian-like prior trained on the Oxford Spires and Newer College datasets, which primarily capture head-mounted pedestrian motion. However, applying this prior to handheld camera data from TUM RGB-D failed to reduce trajectory errors and drift. This suggests that motion dynamics learned from pedestrian or wearable-camera datasets do not transfer well to the more unconstrained, handheld camera movements found in TUM RGB-D. Next, we trained priors directly on TUM RGB-D data. When the prior was trained on sequences excluding the target desk1 sequence, it led to slight improvements in localisation estimates, indicating that even similar handheld sequences can provide limited correction. The strongest effect occurred when we overfitted the motion prior on the desk1 sequence itself, after which applying the correction yielded further modest improvements.

We also compare our results the a baseline from Ma *et al.*, which applies simple temporal smoothing directly on pose estimates. This baseline outperforms our motion prior correction on TUM RGB-D, achieving lower ATE RMSE values. We report additional evaluation metrics like depth estimation error (Depth RMSE, Depth L1), and image reconstruction quality metrics (PSNR, MS-SSIM, LPIPS) in the appendix for all conditions A.1.

We attribute the underperformance of our correction primarily to the backbone model's difficulties in learning reliable motion dynamics from TUM RGB-D. As discussed in Section 4.3.3, the average ATE relative to the 3D distance traversed was around 20%, indicating that the motion prior had difficulties learning the precise patterns of this dataset. This likely stems from the variable motions that are possible with handheld sensors, which make it challenging to learn a strong motion prior purely from pose sequences. Figure 4.8 shows examples of TUM RGB-D trajectories that our backbone model struggled to produce adequate K samples, and hence, an adequate KDE from. As manifested in the spread-out KDE the samples fell into a wide region, and sometimes were offset from the last observed pose.

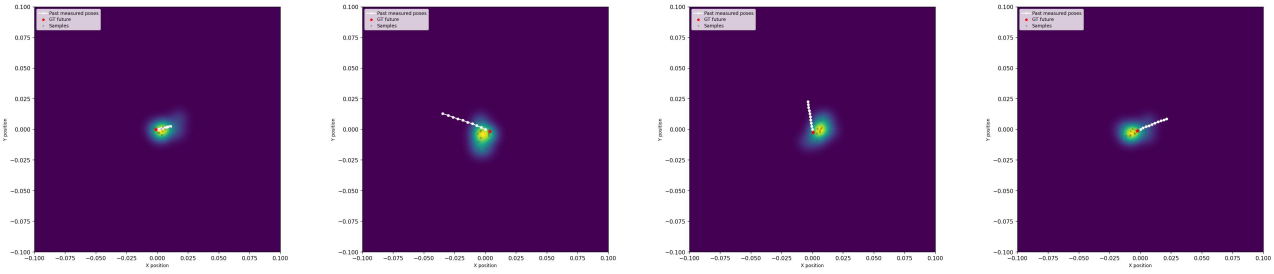


Figure 4.8: Examples of TUM RGB-D trajectories where the backbone model failed to generate correct and narrow K samples, resulting in a wide, and occasionally offset KDE prior and medium correction quality.

Again, we observed that performance is sensitive to the correction parameters bandwidth σ and update stepsize λ, as detailed in Table 4.9. Suboptimal parameters could result in degraded performance. We elaborate on this parameter dependence and potential adaptive solutions in our discussion section.

In summary, although our learned motion prior effectively corrected the *simulated* pose-prediction algorithm on KITTI, it was less successful at refining *SplaTAM* localisations on the

TUM RGB-D dataset. Two reasons explain this discrepancy. First, the simulated algorithm had larger initial errors than SplaTAM's localisation leaving more room for improvement. Second, our backbone model predicted KITTI trajectories significantly better than TUM RGB-D 4.3.3. Hand-held motion patterns are more variable and harder to learn from limited data. Datasets with inherently stronger motion constraints, such as autonomous driving datasets, may yield more robust priors for systems like SplaTAM, which we discuss in our conclusion 5. Lastly, poorer performance of the priors trained on Newer College and Oxford Spires, the results also show that correction performance depends on the compatibility between the prior's motion characteristics and the dynamics of the localisation estimates being adjusted.

Impact of Ground-Truth Start on Motion-Prior Correction

In this experiment, we investigated how initial localisation errors influence the effectiveness of our motion prior correction. We modified the SplaTAM pipeline to start from pose 11, supplying the first ten ground-truth poses as input to our motion prior. This allowed us to begin corrections based on a perfectly accurate past. By contrast, in our standard approach 4.4.2, the first ten localisation estimates produced by SplaTAM are left untouched, and from pose 11 onward, we used these estimated poses (not the ground truth) as input to the motion prior. This setup removed the impact of early SLAM estimation errors on the input to the motion prior - closer to the conditions the backbone was trained on, where predictions relied on noise-free GT inputs.

While overall localisation performance increased, unexpectedly, this approach did not improve the final ATE over the vanilla SplaTAM pipeline. Results were slightly worse or roughly on par with the standard SplaTAM performance using noisy SLAM inputs. This suggests that, in the regular pipeline 4.4.2, SplaTAM already introduced considerable localisation errors within the first ten poses, leading to downstream inconsistencies that our motion prior previously helped to offset.

Condition	λ	σ	**ATE RMSE (cm)**
Existing SplaTAM Variant			
SplaTAM baseline (first 10 GT poses)	—	—	**2.51**
Our Motion-Prior Experiments (GT warm-start)			
Trained on TUM RGB-D excl. desk1	0.005	0.004	2.56
Trained on TUM RGB-D excl. desk1	0.005	0.015	3.00
Trained on TUM RGB-D desk1 (overfit)	0.005	0.004	2.51
Trained on TUM RGB-D desk1 (overfit)	0.005	0.015	2.72

Table 4.10: Final ATE RMSE (cm) for SplaTAM with a ground-truth start (first 10 poses): baseline variant versus motion-prior corrections with $K = 30$. The best result is highlighted in bold.

4.5 Ablations

4.5.1 Performance of Different Rotation Representations

We investigate how different rotation representations affect our model's ability to predict future trajectories. While literature on stochastic pedestrian trajectory prediction focuses on 2D translation prediction [14, 16, 38, 23, 29, 39, 35, 36, 13, 24] we predict full 3D poses, including rotations. This increases the change of potential errors and we need to assess how well different rotation parameterisations support predictions compared to 3D translation alone.

Our experiments show that adding rotation information generally improves translation prediction performance, but the magnitude depends on the dataset and motion type. For structured datasets like KITTI Odometry and our Synthetic Fork dataset including rotation led to better results, with the quaternion representation performing best. In contrast, for Newer College and Oxford Spires (pedestrian datasets), we found that 6D or Lie algebra performed best, outperforming the quaternion representation.

Dataset	**2D Trans.** ATE/ADE/FDE	**3D Trans.** ATE/ADE/FDE
Oxford Spires	0.0789/0.0653/0.1329	0.0952/0.0819/0.1336
Newer College	0.2358/0.1597/0.3476	0.2234/0.1729/0.3028
KITTI Odometry	1.0389/0.6942/1.9198	1.2783/0.8368/2.3099
TUM RGB-D	0.0077/0.0064/0.0148	0.0137/0.0121/0.0245
Synthetic Fork	0.7239/0.2497/0.3947	0.5445/0.2165/0.2781

Table 4.11: Test performance (ATE/ADE/FDE) for translation-only pose representations, predicted 20 timesteps into the future.

Dataset	**Lie** ATE/ADE/FDE	**Quaternion** ATE/ADE/FDE	**6D** ATE/ADE/FDE
Oxford Spires	0.0939/0.0802/0.1515	0.0980/0.0833/0.1548	**0.0913**/**0.0795**/**0.1393**
Newer College	**0.2094**/**0.1663**/**0.3170**	0.2683/0.1819/0.3545	0.2494/0.2072/0.3887
KITTI Odometry	1.2618/0.8337/2.2870	**1.2238**/0.8284/2.3818	1.2747/**0.8123**/**2.2652**
TUM RGB-D	0.0206/0.0190/0.0292	**0.0127**/**0.0109**/**0.0232**	0.0363/0.0302/0.0524
Synthetic Fork	0.4720/0.1852/0.2014	**0.3100**/**0.1407**/**0.1224**	0.8464/0.3570/0.3889

Table 4.12: Test performance (ATE/ADE/FDE) for 3D translation + rotation pose representations (Lie, Quaternion, 6D), predicted 20 timesteps into the future. The best metric is highlighted in bold.

For the TUM RGB-D and other pedestrian datasets, not all rotation representations improved performance compared to translation-only. This could be explained by two factors: firstly, we did not scale our model architecture in size when adding rotation dimensions, possibly limiting its ability to effectively learn complex rotation changes as those of TUM RGB-D. Secondly, unconstrained rotations of handheld motion carry little information about the direction of translation. In datasets captured by a person wearing a head-mounted camera (Oxford Spires) or holding a camera by hand (TUM RGB-D, Newer College), rotation and translation are often decoupled. A pedestrian might look around without changing walking direction, or swing the camera independently of their path. In contrast, in datasets like KITTI, the car's

motion is strongly constrained by its orientation. The direction of the car body, and therefore the roof-mounted sensors, is quite fixed relative to the direction of the wheels which determine the translation direction. If the car's roof is facing forward, the car can only advance or reverse along roughly that same line. Hence, the yaw rotation directly constrains future translation.

Finally, that our results differ per representation illustrates that the strengths and weaknesses inherent to different rotation representations impact performance in practice and has been a topic of research [11]. Lie algebra provides a minimal, interpretable local representation but suffers from discontinuities and singularities near large rotations. The 6D representation avoids such issues by learning an overparameterised embedding, projecting back to valid rotation matrices, but this means it does not live on the SO(3) manifold. Quaternions combine rotation and translation but can be sensitive to small errors and more challenging to optimise [4]. Therefore, the performance of the adding rotation information depends on the interaction between representation and the dataset's motion type, how coupled rotation and translation are, and the model's capacity to leverage these signals with limited parameters.

Chapter 5

Conclusion

In this research, we have developed and analysed a diffusion-based motion prior to denoise SE(3) trajectories in localisation pipelines. Our experiments on both synthetic sequences and real-world RGB-D data demonstrate that multi-modal priors together with a KDE-based correction step, provide drift reduction under certain conditions, addressing a core problem in SLAM systems. In the following, we revisit each of our research questions, summarise the core findings, and discuss their implications for future research.

RQ1: *What are the strengths and limitations of existing stochastic trajectory prediction methods (e.g., LED, MID) for modeling SE(3) motion, and how can they be adapted for pose estimation tasks?*

MID [13] and LED [24] reliably produce diverse, multi-modal future trajectories, as shown in our synthetic fork experiment. LED's learned initialiser accelerates inference, making diffusion models viable for near-real-time use. However, these methods were originally limited to 2D pedestrian motion. We extended them to SE(3) by representing poses in a higher-dimensional embedding 3.1.1 and redesigning the network loss to jointly optimise rotation and translation 3.1.3.

RQ2: *Can we design a pipeline that effectively learns motion estimates for SE(3) trajectories based on input that is noisy or biased?* Yes, our pipeline successfully learns to predict clean, noise-free SE(3) trajectories from biased or noisy past input 4.3.2. Even without access to the true past, the model recovers accurate future motion by implicitly learning to factor out measurement errors. This demonstrates its potential as a robust denoising module, especially when trained on data that reflects known sensor or algorithmic biases.

RQ3: *How does applying the motion prior influence localisation performance in both simulated pose estimation algorithms and live SLAM pipelines such as Splat-SLAM, particularly regarding drift reduction?*

The motion prior effectively reduced drift in our simulated localisation tasks, consistently lowering trajectory errors across conditions. However, corrections in live SLAM pipelines such as Splat-SLAM were more modest. Improvements depended on the compatibility between the prior's learned motion and the SLAM data, as well as on the backbone's ability to model that motion.

RQ4: *Can a diffusion-based motion prior outperform baseline approaches such as the continuous pose function by Ma et al. [22]?*

In theory, our diffusion-based motion prior has a key advantage over baselines like the continuous pose function: it conditions on multiple past poses and learns rich, multimodal motion

dynamics from data. This should allow it to make more informed and flexible corrections, particularly in ambiguous or uncertain motion scenarios. In practice, however, our method did not outperform Ma et al.'s baseline [22] on TUM RGB-D, and was limited by the backbone's ability to learn accurate motion distributions from TUM RGB-D training data.

RQ5: *Which representation of rotation, Lie algebra, quaternions, or 6D continuous representations, leads to the most accurate learning of SE(3) trajectories within the diffusion-based motion prior framework?*

The most effective rotation representation depends on the dataset and underlying motion constraints. For KITTI, TUM RGB-D, and our Synthetic Fork dataset, quaternions performed best. In contrast, for the pedestrian datasets Newer College or Oxford Spires, Lie algebra and 6D representations achieved higher scores. Overall, while all representations have trade-offs, our results suggest that no single method is universally superior. Accuracy is linked to the nature of the motion, the model's capacity, and the representational properties of each approach.

Putting these findings together, we now return to our main research question:

Main Research Question: *Can we design a learned motion prior based on diffusion models that effectively denoises SE(3) trajectory estimates in SLAM pipelines?*

Yes, our results show that it is possible to design a diffusion-based motion prior that learns to denoise SE(3) trajectories by leveraging past pose horizons and capturing multimodal motion patterns. The prior consistently corrected pose estimates in a simulated correction task. However, in the real-world pipeline SplaTAM our approach could only slightly outperform baseline SplaTAM performance. While the method shows potential, its performance was bounded by the backbone's ability to model complex motion dynamics, and by how well the prior aligns with the motion characteristics of the SLAM system being corrected.

Limitations of the Experimental Setup Our experiments were subject to several limitations. First, all SLAM experiments were conducted solely on the TUM RGB-D dataset, despite our backbone model showing weaker performance on this data due to its highly unconstrained, handheld motion dynamics. Second, the relatively small size of both the prior training datasets 4.2 limited the motion patterns the model could learn. Finally, the correction procedure required manual tuning of sensitive hyperparameters - specifically the KDE bandwidth and correction stepsize - instead of adaptive parameter selection.

5.1 Future Directions

5.1.1 Datasets with Stronger Motion Constraints

One key limitation of our evaluation is that SLAM experiments were conducted exclusively on TUM RGB-D, despite our model demonstrating stronger performance on structured datasets like KITTI in simulated settings. Future work should evaluate motion priors on datasets with stronger motion constraints, such as autonomous driving datasets (e.g., KITTI [10], Argoverse [34] or nuScenes [5]. Driving motion patterns follow predictable acceleration, and have constrained rotations, just as minimal variance along the z-coordinate. The motion prior could learn much more robust motion statistics which in turn are better suited to correct noise and drift in localisation estimates, like those in SLAM pipelines. In our experiments, the KITTI dataset already illustrated this: our backbone model was able to learn and predict KITTI's structured car motion trajectories with relatively low ATE, especially compared to TUM RGB-D.

5.1.2 In-Pipeline Motion Prior Regularisation

Instead of implementing the motion prior as a post-prediction corrector, future work could explore integrating it directly into the optimisation loop of differentiable SLAM pipelines. By introducing a prior-based loss term into the SLAM objective, the system optimises for both photometric consistency and plausible motion dynamics. This produces a different loss landscape and may improve convergence, although it does not guarantee better performance.

The total SLAM objective becomes:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{SLAM}} + \lambda \cdot \mathcal{L}_{\text{prior}}, \tag{5.1}$$

where the prior loss term $\mathcal{L}_{\text{prior}}$, weighted by lambda, encourages adhering to motion patterns learned via our prior.

Despite these open directions, our results offer a solid foundation for continued research, demonstrating the potential of generative motion models within localisation pipelines. As future work explores datasets with more structured and representative motion patterns, diffusion-based motion priors may evolve into a powerful and generalisable component for improving robust localisation.

Appendix A

Appendix

A.0.1 SplaTAM Correction Full Metrics

Condition	λ	σ	ATE RMSE (cm)	PSNR	Depth RMSE (cm)	Depth L1 (cm)	MS-SSIM	LPIPS
Existing SplaTAM Variants								
SplaTAM baseline	—	—	3.58	21.66	3.11	3.11	0.850	0.232
SplaTAM w/ PoseNet [22]	—	—	**2.99**	18.75	2.47	2.47	0.728	0.348
Our Motion-Prior Experiments								
Trained on TUM RGB-D excl. desk 1	0.001	0.04	3.54	21.15	3.06	3.06	0.838	0.244
Trained on TUM RGB-D excl. desk 1	0.005	0.04	3.52	21.50	2.69	2.69	0.846	0.231
Trained on TUM RGB-D excl. desk 1	0.005	0.15	4.03	20.06	2.42	2.42	0.812	0.262
Trained on TUM RGB-D desk 1 (overfit)	0.005	0.15	3.58	22.59	2.52	2.52	0.890	0.184
Trained on TUM RGB-D desk 1 (overfit)	0.005	0.04	3.49	23.02	2.53	2.53	0.883	0.192

Table A.1: Full quantitative metrics for SplaTAM variants and our motion-prior corrections on Freiburg desk 1 (TUM RGB-D) 4.4.2. Existing variants are listed first, followed by our experiments.

A.0.2 Ground-Truth Start Experiment: SplaTAM Correction Full Metrics

Condition	λ	σ	ATE RMSE (cm)	PSNR	Depth RMSE (cm)	Depth L1 (cm)	MS-SSIM	LPIPS
Existing SplaTAM Variant								
SplaTAM baseline (start at pose 11)	—	—	**2.51**	20.90	7.61	7.61	0.766	0.286
Our Motion-Prior Experiments (start at pose 11)								
Trained on TUM RGB-D excl. desk 1	0.005	0.04	2.56	20.58	6.95	6.95	0.769	0.272
Trained on TUM RGB-D excl. desk 1	0.005	0.15	3.00	19.06	6.48	6.48	0.713	0.324
Trained on TUM RGB-D desk 1 (overfit)	0.005	0.04	2.51	20.15	6.76	6.76	0.752	0.299
Trained on TUM RGB-D desk 1 (overfit)	0.005	0.15	2.72	19.56	6.56	6.56	0.736	0.300

Table A.2: Full quantitative metrics for the Ground-Truth Start on Motion-Prior Correction experiments 4.4.2: baseline SplaTAM versus our motion-prior corrections with $K = 30$.

A.0.3 Performance of Pedestrian Trajectory Prediction Models

Subset	Social-GAN [15]	NMMP [16]	STAR [38]	PECNet [23]	Trajectron++ [29]	Agentformer [39]	MemoNet [36]	NPSN [3]	GroupNet [35]	MID [13]	Leapfrog [24]
ETH	0.87/1.62	0.61/1.08	**0.36/0.65**	0.54/0.87	0.61/1.02	0.45/0.75	0.40/0.61	0.40/0.76	0.46/0.73	0.39/0.66	0.39/0.58
Hotel	0.67/1.37	0.33/0.63	**0.17/0.36**	0.18/0.24	0.19/0.28	0.14/0.22	0.11/0.17	0.12/0.18	0.15/0.25	0.13/0.22	**0.11/0.17**
Univ	0.76/1.52	0.52/1.11	0.31/0.62	0.35/0.60	0.30/0.54	0.25/0.45	0.24/0.43	0.22/0.41	0.26/0.49	0.22/0.45	**0.26/0.43**
Zara1	0.35/0.68	0.32/0.66	0.29/0.52	0.22/0.39	0.24/0.42	0.18/0.30	0.18/0.32	0.17/0.31	0.21/0.39	0.17/0.30	**0.18/0.26**
Zara2	0.42/0.84	0.43/0.85	0.22/0.46	0.17/0.30	0.18/0.32	0.14/0.24	0.14/0.24	0.12/0.24	0.17/0.33	0.13/0.27	**0.13/0.22**

Table A.3: Comparison with baseline models on ETH-UCY dataset. $\text{minADE}_{20}/\text{minFDE}_{20}$ (meters) are reported. **Bold** font represent the best result. In most subsets, Leapfrog [24] achieves the best or second-best performance in ADE/FDE.

Bibliography

[1] Alexandre Alahi, Kratarth Goel, Vignesh Ramanathan, Alexandre Robicquet, Li Fei-Fei, and Silvio Savarese. Social lstm: Human trajectory prediction in crowded spaces. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 961–971, 2016.

[2] Sean Anderson and Timothy D Barfoot. Full steam ahead: Exactly sparse gaussian process regression for batch continuous-time trajectory estimation on se (3). In *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 157–164. IEEE, 2015.

[3] Inhwan Bae, Jin-Hwi Park, and Hae-Gon Jeon. Non-probability sampling network for stochastic human trajectory prediction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6477–6487, 2022.

[4] José Luis Blanco-Claraco. A tutorial on

$$\backslash$$

mathbf {SE}(3) $transformationparameterizationsandon - manifoldoptimization.arXivpreprintarXiv$: 2103.15980, 2021.

[5] Holger Caesar, Varun Bankiti, Alex H. Lang, Sourabh Vora, Venice Erin Liong, Qiang Xu, Anush Krishnan, Yu Pan, Giancarlo Baldan, and Oscar Beijbom. nuscenes: A multimodal dataset for autonomous driving, 2020.

[6] Carlos Campos, Richard Elvira, Juan J Gómez Rodríguez, José MM Montiel, and Juan D Tardós. Orb-slam3: An accurate open-source library for visual, visual–inertial, and multimap slam. *IEEE transactions on robotics*, 37(6):1874–1890, 2021.

[7] Carlos Campos, Richard Elvira, Juan J Gómez Rodríguez, José MM Montiel, and Juan D Tardós. Orb-slam3: An accurate open-source library for visual, visual–inertial, and multimap slam. *IEEE transactions on robotics*, 37(6):1874–1890, 2021.

[8] Ziyu Cao, William Talbot, and Kailai Li. Resple: Recursive spline estimation for lidar-based odometry, 2025.

[9] Xiang Gao, Rui Wang, Nikolaus Demmel, and Daniel Cremers. Ldso: Direct sparse odometry with loop closure. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2198–2204. IEEE, 2018.

[10] A Geiger, P Lenz, C Stiller, and R Urtasun. Vision meets robotics: The KITTI dataset. *The International Journal of Robotics Research*, 32(11):1231–1237, August 2013.

[11] A. René Geist, Jonas Frey, Mikel Zhobro, Anna Levina, and Georg Martius. Learning with 3d rotations, a hitchhiker's guide to so(3), 2024.

[12] Ian J Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. *Advances in neural information processing systems*, 27, 2014.

[13] Tianpei Gu, Guangyi Chen, Junlong Li, Chunze Lin, Yongming Rao, Jie Zhou, and Jiwen Lu. Stochastic trajectory prediction via motion indeterminacy diffusion. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 17113–17122, 2022.

[14] Agrim Gupta, Justin Johnson, Li Fei-Fei, Silvio Savarese, and Alexandre Alahi. Social gan: Socially acceptable trajectories with generative adversarial networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2255–2264, 2018.

[15] Agrim Gupta, Justin Johnson, Li Fei-Fei, Silvio Savarese, and Alexandre Alahi. Social gan: Socially acceptable trajectories with generative adversarial networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2255–2264, 2018.

[16] Yue Hu, Siheng Chen, Ya Zhang, and Xiao Gu. Collaborative motion prediction via neural motion message passing. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 6319–6328, 2020.

[17] Nikhil Keetha, Jay Karhade, Krishna Murthy Jatavallabhula, Gengshan Yang, Sebastian Scherer, Deva Ramanan, and Jonathon Luiten. Splatam: Splat track & map 3d gaussians for dense rgb-d slam. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 21357–21366, 2024.

[18] Yaron Lipman, Ricky TQ Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2022.

[19] Mingyu Liu, Ekim Yurtsever, Jonathan Fossaert, Xingcheng Zhou, Walter Zimmer, Yuning Cui, Bare Luka Zagar, and Alois C Knoll. A survey on autonomous driving datasets: Statistics, annotation quality, and a future outlook. *IEEE Transactions on Intelligent Vehicles*, 2024.

[20] Wenxin Liu, David Caruso, Eddy Ilg, Jing Dong, Anastasios I Mourikis, Kostas Daniilidis, Vijay Kumar, and Jakob Engel. Tlio: Tight learned inertial odometry. *IEEE Robotics and Automation Letters*, 5(4):5653–5660, 2020.

[21] Mingyang Lyu, Yibo Zhao, Chao Huang, and Hailong Huang. Unmanned aerial vehicles for search and rescue: A survey. *Remote Sensing*, 15(13), 2023.

[22] Qi Ma, Danda Pani Paudel, Ajad Chhatkuli, and Luc Van Gool. Continuous pose for monocular cameras in neural implicit representation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 5291–5301, 2024.

[23] Karttikeya Mangalam, Harshayu Girase, Shreyas Agarwal, Kuan-Hui Lee, Ehsan Adeli, Jitendra Malik, and Adrien Gaidon. It is not the journey but the destination: Endpoint conditioned trajectory prediction. In *European conference on computer vision*, pages 759–776. Springer, 2020.

[24] Weibo Mao, Chenxin Xu, Qi Zhu, Siheng Chen, and Yanfeng Wang. Leapfrog diffusion model for stochastic trajectory prediction. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 5517–5526, 2023.

[25] Raul Mur-Artal, Jose Maria Martinez Montiel, and Juan D Tardos. Orb-slam: A versatile and accurate monocular slam system. *IEEE transactions on robotics*, 31(5):1147–1163, 2015.

[26] Raul Mur-Artal and Juan D Tardós. Orb-slam2: An open-source slam system for monocular, stereo, and rgb-d cameras. *IEEE transactions on robotics*, 33(5):1255–1262, 2017.

[27] Milad Ramezani, Yiduo Wang, Marco Camurri, David Wisth, Matias Mattamala, and Maurice Fallon. The newer college dataset: Handheld lidar, inertial and vision with ground truth. In *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, October 2020.

[28] Danilo Rezende and Shakir Mohamed. Variational inference with normalizing flows. In *International conference on machine learning*, pages 1530–1538. PMLR, 2015.

[29] Tim Salzmann, Boris Ivanovic, Punarjay Chakravarty, and Marco Pavone. Trajectron++: Dynamically-feasible trajectory forecasting with heterogeneous data. In *European Conference on Computer Vision*, pages 683–700. Springer, 2020.

[30] Kihyuk Sohn, Honglak Lee, and Xinchen Yan. Learning structured output representation using deep conditional generative models. *Advances in neural information processing systems*, 28, 2015.

[31] J. Sturm, N. Engelhard, F. Endres, W. Burgard, and D. Cremers. A benchmark for the evaluation of rgb-d slam systems. In *Proc. of the International Conference on Intelligent Robot Systems (IROS)*, Oct. 2012.

[32] Yifu Tao, Miguel Ángel Muñoz-Bañón, Lintong Zhang, Jiahao Wang, Lanke Frank Tarimo Fu, and Maurice Fallon. The oxford spires dataset: Benchmarking large-scale lidar-visual localisation, reconstruction and radiance field methods, 2024.

[33] Shinji Umeyama. Least-squares estimation of transformation parameters between two point patterns. *IEEE Transactions on pattern analysis and machine intelligence*, 13(4):376–380, 2002.

[34] Benjamin Wilson, William Qi, Tanmay Agarwal, John Lambert, Jagjeet Singh, Siddhesh Khandelwal, Bowen Pan, Ratnesh Kumar, Andrew Hartnett, Jhony Kaesemodel Pontes, Deva Ramanan, Peter Carr, and James Hays. Argoverse 2: Next generation datasets for self-driving perception and forecasting, 2023.

[35] Chenxin Xu, Maosen Li, Zhenyang Ni, Ya Zhang, and Siheng Chen. Groupnet: Multiscale hypergraph neural networks for trajectory prediction with relational reasoning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6498–6507, 2022.

[36] Chenxin Xu, Weibo Mao, Wenjun Zhang, and Siheng Chen. Remember intentions: Retrospective-memory-based trajectory prediction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 6488–6497, 2022.

[37] Meng Xu, Qiqi Shu, Zhao Huang, Guang Chen, and Stefan Poslad. Arlo: Augmented reality localization optimization for real-time pose estimation and human–computer interaction. *Electronics*, 14(7), 2025.

[38] Cunjun Yu, Xiao Ma, Jiawei Ren, Haiyu Zhao, and Shuai Yi. Spatio-temporal graph transformer networks for pedestrian trajectory prediction. In *European conference on computer vision*, pages 507–523. Springer, 2020.

[39] Ye Yuan, Xinshuo Weng, Yanglan Ou, and Kris M Kitani. Agentformer: Agent-aware transformers for socio-temporal multi-agent forecasting. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 9813–9823, 2021.

[40] Cheng Zhao, Weixing Mei, and Wei Pan. Building a grid-semantic map for the navigation of service robots through human–robot interaction. *Digital Communications and Networks*, 1(4):253–266, 2015.